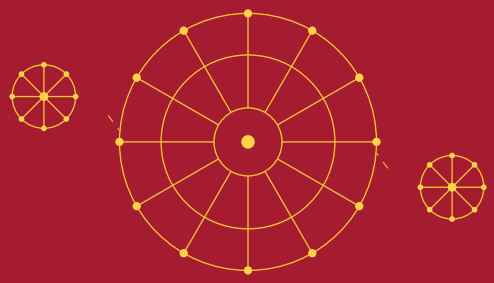


An Embedded-Database Monograph

ChakraDB

The Definitive Guide

*The embedded **HTAP** database with built-in graph capabilities — write, analyze, and traverse live data.*

The ChakraDB Authors

Architecture · Algorithms · Graph · Comparisons • Working Draft, 2026

Contents

Part I

Preface

Chapter 1

About This Book

If I have seen further it is by standing on the shoulders of Giants.

— *Isaac Newton*

This is the complete guide to **ChakraDB** — an embedded HTAP database with built-in graph capabilities. It is written for three audiences at once, and it is organized so each can find its level:

- **Users** who want to build applications — start at Part V (*Getting Started*) and Part VI (*Developer Guide & Tutorials*).
- **Architects** evaluating the engine — read Part I (*Introduction*), Part II (*Architecture*), and Part IX (*Comparative Studies*).
- **Contributors and the curious** who want to know *how* it works down to the algorithm — Part III (*Algorithms*) is the deep end.

1.1 How the book is organized

Part	What it covers
I — Introduction	What ChakraDB is, the problem it solves, its design principles and cost model.
II — Architecture	The engine's structure: storage, MVCC, durability, compaction, the query router.
III — Algorithms	Each core algorithm in detail, with pseudocode and complexity.
IV — Graph	The property-graph layer: adjacency, CSR snapshots, and graph algorithms.
V — Getting Started	Install, first database, Python, first graph.
VI — Developer Guide	SQL, types, constraints, transactions, COPY, backup, observability, tutorials.

Part	What it covers
VII — Operations	Durability tuning, limits, monitoring, disaster recovery.
VIII — Case Studies	Worked end-to-end applications.
IX — Comparative Studies	Head-to-head with DuckDB, SQLite, Neo4j, PostgreSQL, with methodology.
X — Reference	The Rust and Python APIs, configuration, errors, file formats.
Appendices	Glossary, design-decision log, bibliography.

1.2 Conventions

- **Diagrams** are rendered with Mermaid. Every architecture and algorithm chapter has at least one.
- **Code** is real. Rust snippets compile against the public API; SQL runs on the shipped engine.
- **Callouts** flag the important asymmetries: > This is a *design commitment* — a place where ChakraDB deliberately pays a > cost so that it can be cheap somewhere that matters more.

1.3 Relationship to the source docs

This book synthesizes and expands the design documents that live alongside the code — `requirements.md` (the architecture specification), `sql-reference.md`, `clickbench-findings.md`, and the milestone records. Where the book and a source doc disagree, the code is the arbiter; the book aims to track it.

1.4 Status

ChakraDB is a working engine — durable, crash-tested, benchmarked against DuckDB — with an actively evolving surface. Chapters that describe not-yet-shipped work (for example, parts of the graph roadmap) say so explicitly.

Part II

Introduction

Chapter 2

Introduction to ChakraDB

No man ever steps in the same river twice, for it is not the same river and he is not the same man.

— *Heraclitus*

Most databases are built for data that has stopped moving. You load it, then you query it. ChakraDB is built for the opposite premise — the one Heraclitus noticed twenty-five centuries ago: **the data is always changing, and the interesting question is what is true of it *now*, while it is still arriving.** Everything in this book follows from taking that premise seriously.

2.1 The one-sentence summary

ChakraDB is an embedded database that serves a continuous stream of durable writes while running analytical queries and graph traversals that never block — over the same live data, in one process, in an open columnar format.

Unpack that sentence and you have the whole system:

- *embedded* — a library linked into your process, like SQLite or DuckDB, not a server you operate.
- *continuous durable writes* — ACID, WAL-backed, and **concurrent**: many writers at once.
- *analytical queries and graph traversals that never block* — readers see a consistent snapshot and are never stopped by a writer.
- *the same live data* — no ETL, no second system, no staleness between the operational store and the analytical one.
- *open columnar format* — data lives on disk as Apache Arrow, readable by any Arrow tool.

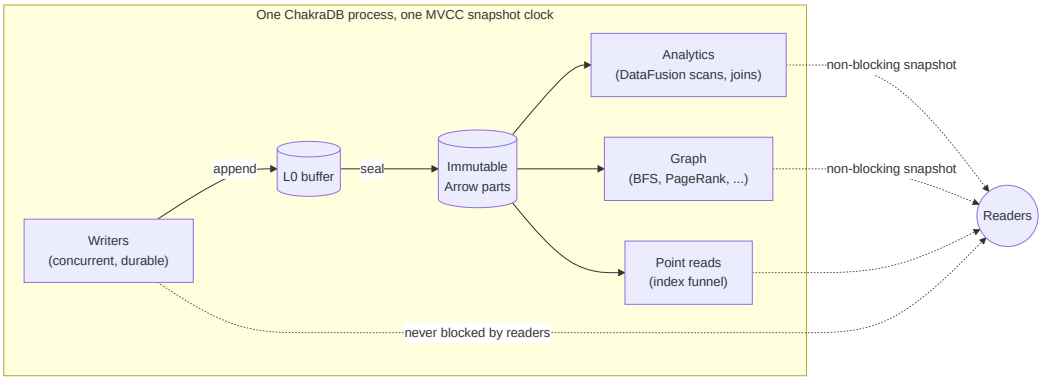
2.2 What makes it different

Every database chooses a shape of work to be excellent at, and pays for that choice somewhere else. ChakraDB's choice is the *seam* between transactional and analytical work, extended to graph — the place most systems refuse to stand.

Consider where the competition sits:

- **DuckDB** gives blazing analytics over data you loaded earlier. But it holds a **single-writer file lock** — a second writer is refused at the OS level (`IO Error: Could not set lock`). It is superb at static analytics and structurally unable to serve a live write stream.
- **SQLite** is a rock-solid embedded transactional store, but its analytics are row-at-a-time, and one writer serializes the entire database.
- **Neo4j** gives graph traversals, but it is a server, and mutation takes locks that contend with the reads doing the traversal.
- **NetworkX / igraph** give graph algorithms — over a *dead static copy* you loaded into memory, stale the instant it is built.

ChakraDB's wager is that the valuable modern workload is precisely the one that falls *between* these: **data that is still arriving, that you want to analyze and traverse at the same time.** That is HTAP — Hybrid Transactional/Analytical Processing — plus graph, in an embedded library.



The differentiator is **concurrency, not raw scan speed.** ChakraDB does not aim to out-scan DuckDB on a static file; it aims to serve a workload DuckDB *structurally cannot* — concurrent writers with non-blocking analytical and graph reads.

2.2.1 The four properties, at once

Individually, existing engines have three of the four properties below. The gap ChakraDB targets is having all four together:

	Embedded	ACID + MVCC	Concurrent writes + non-blocking scans	Open on-disk format
DuckDB			× single writer	via extensions
SQLite			× one writer	×
Neo4j	× server		lock contention	×
ArcticDB		×		
ChakraDB				Arrow IPC parts

2.2.2 ...and now, graph

On top of that HTAP core, ChakraDB adds a **built-in graph layer** — not a bolted-on second store, but a thin layer that reuses exactly the properties above. A graph edge whose key encodes `(src, dst)` is stored *sorted by source*, so a node's neighbors are a contiguous key range (clustered adjacency for free); a graph algorithm builds its working set from **one MVCC snapshot**, so it runs to completion over a consistent graph while edges keep streaming in (live graph analytics); and sorted-by-source edges are already grouped by source, so building the CSR representation every algorithm wants is a single linear scan (the storage format is the algorithm's input format). The result is a client experience where BFS, shortest path, PageRank, connected components, and triangle counting are one method call away, over data you are still writing to.

2.3 The problem: why another database?

The case for ChakraDB is a case against **fragmentation**. The modern data stack has splintered a single logical need — "know what is true of my live data" — across three or four systems: an OLTP store for the writes, a warehouse for the analytics, a graph database for the relationships, and an ETL fabric to keep them approximately in sync. Each hop adds latency, staleness, operational surface, and a consistency model to reconcile.

The fragmentation exists because of a genuine technical tension — the shapes of work want opposite physical layouts:

- **Transactional** work wants row-at-a-time mutation, point lookups, and a durable log. It is latency-bound and write-heavy.
- **Analytical** work wants columnar, compressed, vectorized scans over large row counts. It is throughput-bound and read-heavy.
- **Graph** work wants pointer-chasing adjacency and whole-graph iteration.

For decades the accepted wisdom — Stonebraker's "one size does not fit all" — was that these needs demand separate engines. ChakraDB does not dispute that a single *physical layout* cannot serve all three. It disputes that they need separate *systems*. The insight is that a **log-structured, Arrow-native store with MVCC** can present the right layout to each: a row buffer and a sorted key index for the transactional path, immutable columnar parts for the analytical path, and sorted-by-source edges for the graph path — all over **one snapshot clock**, so a reader of any kind sees one consistent instant while writers proceed unblocked.

The specific thing that becomes possible, that no single embedded engine offered before, is: **ingest continuously, and analyze and traverse the result at the same time, with no ETL and**

no lock fight. The case studies show why that is not a luxury — for real-time fraud scoring, live recommendations, and streaming dashboards, the staleness and the lock contention of the fragmented stack are the whole problem.

2.4 Design principles

Five commitments shape every decision in this book. They are worth stating up front because when a trade-off arises, these are how it is resolved.

1. **Concurrency is the wedge.** ChakraDB competes on serving writes-plus-analytics-plus-graph on live data — a workload DuckDB structurally cannot — *not* on out-scanning it on a static file. When a choice trades a little cold-scan speed for a lot of live-concurrency, it takes that trade.
2. **Buy execution, build storage.** The vectorized analytical engine is a solved, multi-year problem; ChakraDB *buys* it (Apache DataFusion) and spends its own effort where the differentiation is — storage, MVCC, durability, the transactional interpreter, and the graph layer. (See The Query Layer.)
3. **Cold data pays nothing.** The cost of concurrency is borne only by recently-modified data. A part that predates every live snapshot is scanned with no per-row version check at all — so a billion-row table with a thousand recent mutations pays version costs on a thousand rows, not a billion. (See MVCC.)
4. **Immutability over invalidation.** Sealed parts are never edited in place; updates and deletes are new versions and deletion-vector marks. Immutable data is safe to share lock-free with any number of readers, is trivially cacheable, and makes an open on-disk format possible.
5. **Publish the harness, not just the number.** No neutral benchmark measures the concurrency axis ChakraDB competes on, so every performance claim in this book ships with the code that produced it (Part IX). A number without a reproducible source is a hypothesis, not a measurement.

2.5 The cost model

Fast writes, fast scans, and high concurrency are in genuine tension; every system claiming all three has chosen *where to absorb the cost*. Stating ChakraDB's choice explicitly:

The goal	The cost it creates	Where ChakraDB pays it
Fast writes	data accumulates as unmerged parts → scans slow	**compaction** (a designed, back-pressured subsystem), never the read path
Fast scans	needs merged, sorted, columnar data → write amplification	absorbed by compaction, off the write path
High concurrency	needs versioning → readers may pay version cost per row	paid **only** by recently-modified data; cold parts pay `O(1)`

The goal	The cost it creates	Where ChakraDB pays it
Open on-disk format	needs a committed format + manifest → commit overhead	Arrow IPC parts + a manifest; publication cadence is a tunable knob

Three absorption points define the engine:

- **The cost of fast writes is paid by background-free, caller-driven compaction** — and if compaction cannot keep up, the engine applies *explicit* ingest backpressure rather than silently degrading scans. This is a hard commitment, not an aspiration.
- **The cost of concurrency is paid only by recently-modified data** — the MVCC fast path (principle 3).
- **The cost of the open format is paid in visibility latency, not write latency** — internal transactions commit fast to a local log; the durable columnar state is materialized on a separate cadence.

If any of these trades is unacceptable for your workload, the architecture is the wrong fit — so they are stated here, at the front, to be challenged.

2.6 Who uses ChakraDB

The engine fits workloads that are simultaneously *operational* and *analytical*, on a single node, embedded in an application:

- **Real-time fraud and risk** — score transactions as they arrive, and traverse the entity graph they form, over one consistent view.
- **Live recommendations** — personalized PageRank / common-neighbors over a graph that updates continuously.
- **Streaming dashboards** — ingest events and serve aggregations without an ETL hop or a read/write lock fight.
- **Exact-money ledgers** — `DECIMAL` arithmetic that never rounds, with ACID transactions.

2.7 The technology stack

ChakraDB is written in Rust; the core forbids `unsafe` code. Data is Apache **Arrow** end to end — in memory, on disk (Arrow IPC), and across the DataFusion boundary (zero-copy). Vectorized analytical execution is **bought** from Apache DataFusion; storage, MVCC, durability, the transactional interpreter, and the graph layer are **built**. The next part, Architecture, shows how those pieces fit, and why the "buy execution, build storage" split is the whole strategy.

Part III

The Engine

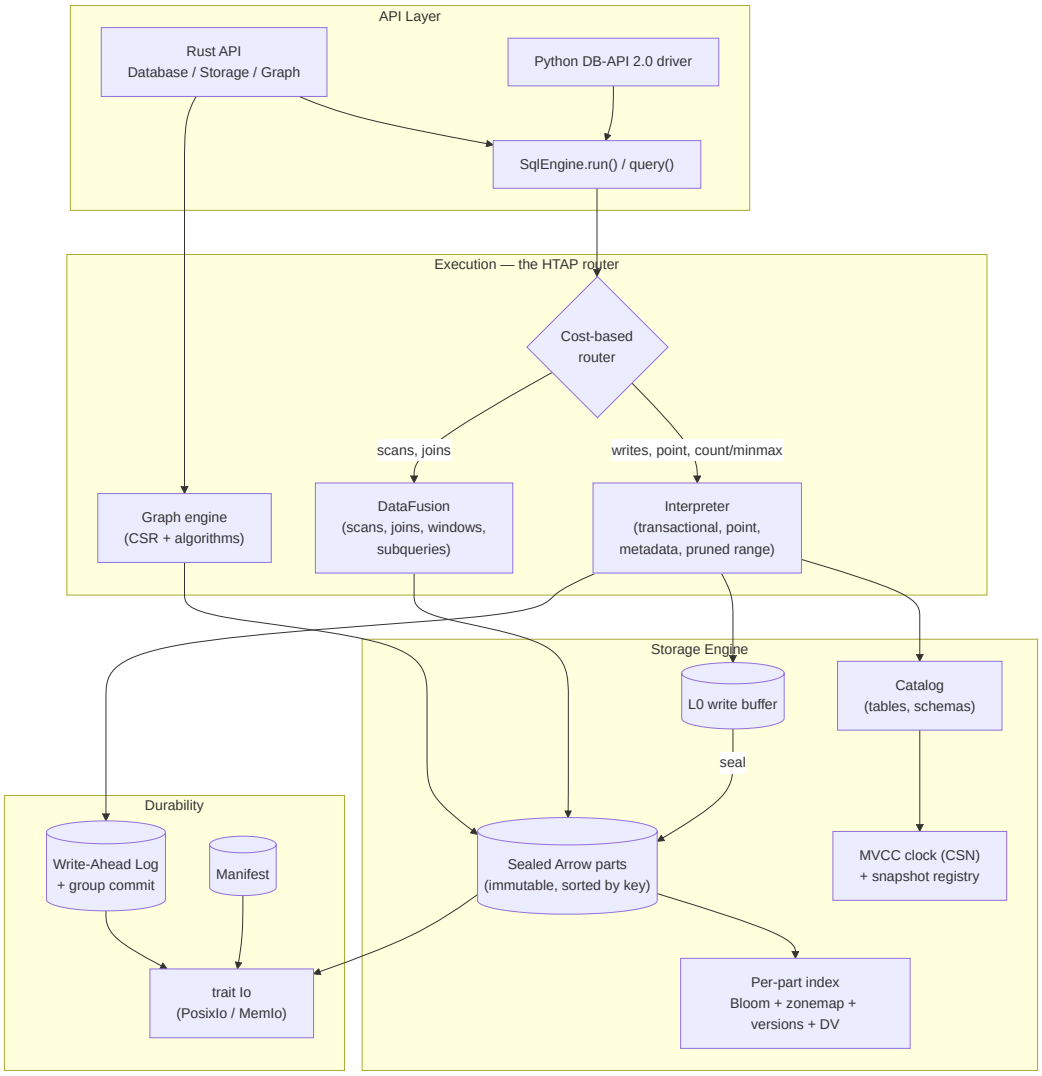
Chapter 3

System Overview

Architecture is the decisions that you wish you could get right early in a project.

— *Ralph Johnson*

ChakraDB is a stack of narrow layers with one guiding split: **buy execution, build storage.** The vectorized analytical engine is Apache DataFusion; the storage engine, MVCC, durability, transactional interpreter, and graph layer are ChakraDB's own. The layers meet at deliberately thin interfaces.



3.1 The layers, top to bottom

API. Three entry points: the Rust `Database`/`Storage`/`Graph` types, a standards-based Python DB-API 2.0 (PEP 249) driver, and the SQL front end (`SqlEngine`).

Execution — the HTAP router. Every SQL statement is routed to whichever engine fits its shape. Cheap transactional shapes — writes, key point-lookups, metadata `COUNT(*)`/`MIN`/`MAX`, and selective range scans — run on a hand-written **interpreter** that owns the write path. Analytical

shapes — large scans, joins, windows, subqueries — run on **DataFusion**. The **graph engine** is a third consumer, building CSR working sets from the same parts. See The HTAP Router.

Storage Engine. A three-tier log-structured store: an in-memory **L0** buffer absorbs writes at memory speed; when it fills it is sealed, sorted by key, into an immutable columnar **Arrow part**; parts accumulate and are periodically merged by compaction. Every part carries a small resident **index** — a Bloom filter, per -column min/max zonemaps, version stamps, and a deletion vector. See The Storage Engine.

Durability. A write-ahead log with group commit makes writes durable before they are acknowledged; a manifest records the catalog and the set of live parts; recovery replays the WAL past the last checkpoint. Everything reaches disk through a single `trait Io`, which has a real POSIX implementation and an in-memory fault -injecting one for tests. See Durability.

3.2 The two invariants that make it fast

Two design invariants recur throughout the architecture, and they are worth holding in mind before the details:

> **1. The sorted key column *is* the index.** Because a sealed part is written sorted by its key, a row's ordinal position *is* its offset. There is no separate key→location map to store or maintain — the index cost is ~1.25 bytes per row and flat with table size. This is the primary-key index.
>
> **2. Cold data pays zero concurrency cost.** MVCC visibility is a pure function of a snapshot number and a part's version stamps. A part that predates every live snapshot is "fully visible" and is scanned with no per-row version check at all. A billion-row table with a thousand recent mutations pays version costs on a thousand rows, not a billion. This is MVCC.

3.3 The data flow of a write and a read

A **write** appends to the WAL (durable), updates the L0 buffer and the MVCC clock, and acknowledges — all without touching any reader. A **read** takes a snapshot number, then either funnels through the index to a single row (point lookup) or scans the parts visible to that snapshot (analytical / graph), holding no lock that a writer could contend on. The following chapters trace both paths in detail.

Chapter 4

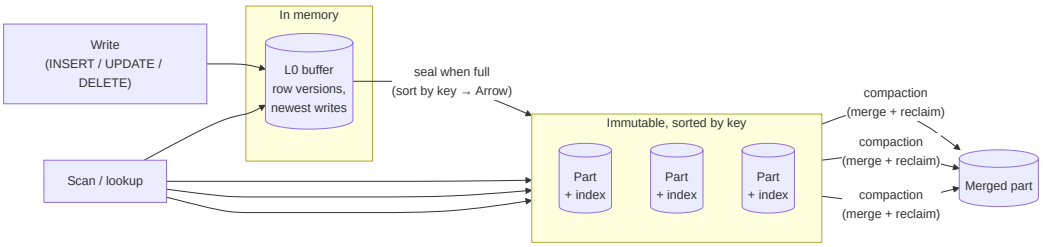
The Storage Engine

Show me your flowcharts and conceal your tables, and I shall continue to be mystified. Show me your tables, and I won't usually need your flowcharts; they'll be obvious.

— *Fred Brooks*

ChakraDB's storage is **log-structured** and **Arrow-native**. Writes land in memory and are sealed into immutable columnar parts; reads see a merged view across the tiers. This chapter is the shape of the store; the algorithms that run over it — visibility, merge, pruning — get their own chapters in Part III.

4.1 Three tiers



1. **L0 — the write buffer.** New row versions land here at memory speed. L0 is a small in-memory structure keyed for point lookup and range scan. It holds the newest, not-yet-sealed writes.

2. **Sealed parts — the columnar body.** When L0 fills (a configurable threshold), it is **sealed**: its rows are sorted by the table's key and written as an immutable **Apache Arrow** record batch. On the durable path a part persists as an Arrow **IPC** stream — an open format any

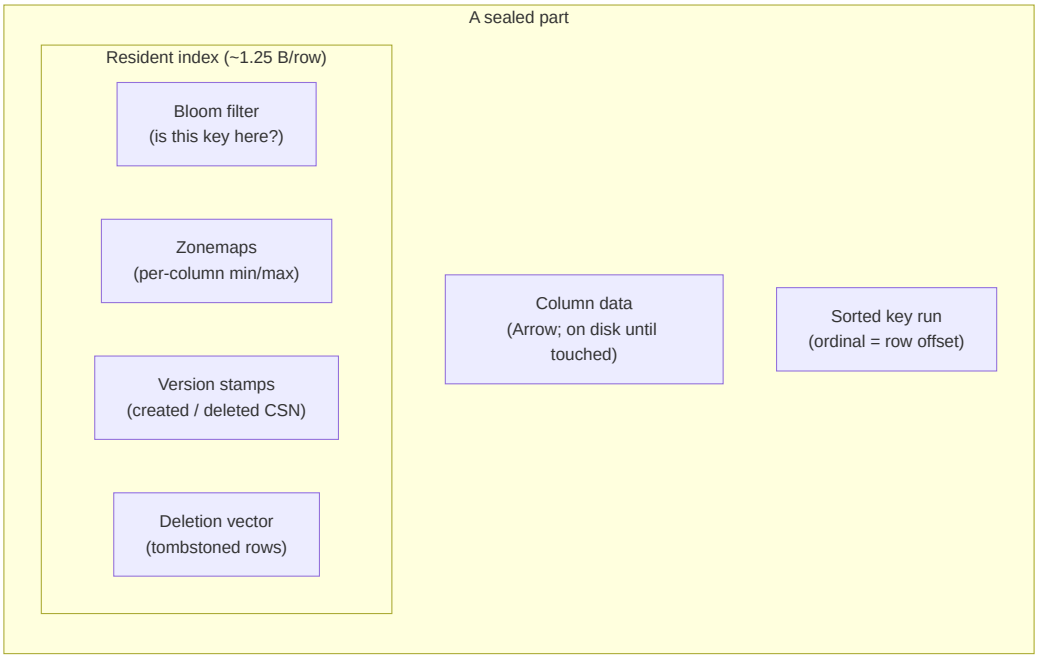
Arrow reader can open. A part never changes after sealing; updates and deletes are expressed as *new* versions and *deletion-vector* marks, not in-place edits.

3. **Compaction — keeping scans fast.** Parts accumulate as writes flow. A background-free, caller-driven **compaction** merges parts, drops rows no live snapshot can see, and collapses version stamps — trading write amplification for scan speed. See Compaction.

The absorption point. Fast writes create unmerged deltas that slow scans; ChakraDB pays that debt in **compaction**, not in the read path, and applies explicit **backpressure** if compaction cannot keep up — never silent scan degradation.

4.2 What a part carries: the resident index

The reason ChakraDB can keep row *data* on disk but stay fast is that each part keeps a small **index resident in memory** — about **1.25 bytes per row**, flat with table size:



- **Bloom filter** — answers "could key k be in this part?" with no disk touch, so a point lookup skips parts that certainly lack the key.
- **Zonemaps** — per-column `(min, max)`. A `WHERE` range or a graph adjacency scan skips any part whose range cannot overlap. This is zonemap pruning.
- **Version stamps** — the `created`/`deleted` CSN per row (or one uniform stamp when the whole part shares one), the input to MVCC visibility.
- **Deletion vector** — which ordinals are tombstoned, so a scan skips deleted rows without rewriting the part.
- **The sorted key run** *is* the index: because the part is sorted by key, the ordinal position is the

row offset, and a lookup is a Bloom probe plus a binary search — no separate key→location map exists. See The Primary-Key Index.

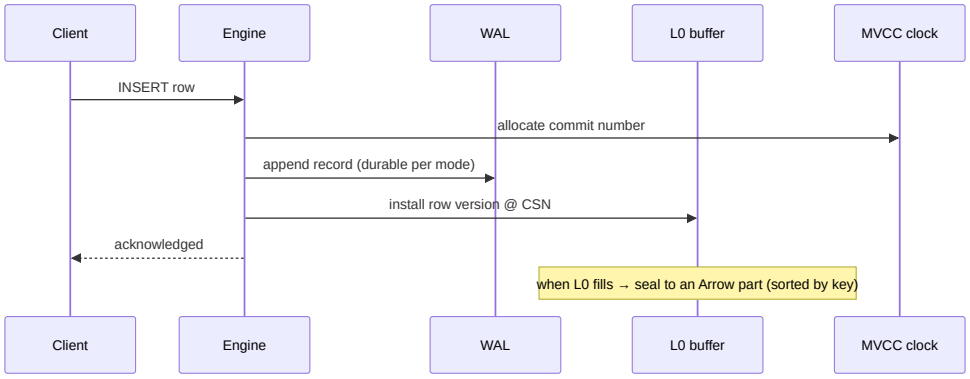
4.3 Arbitrary schemas, any-type keys

A table has any number of columns of any supported type. Its **primary key** is a single column of any type (integer, text, float, boolean, date, decimal), or — if no key is declared — a hidden auto-increment `_rowid` (a *keyless* table). One idea keeps the engine simple: **every table has exactly one key column**; "keyless" is just a table whose key is hidden.

4.4 Why not a general buffer pool?

Parts are immutable and whole-part granular, so ChakraDB deliberately has **no LRU, no page-replacement cache**. The policy is "fault a part's data in on first touch, keep it until the part is dropped." That is the honest amount of machinery for immutable parts — and it is why the *resident index*, not disk, is the scaling ceiling (see Limits).

4.5 The write path in one picture



Notice what the write path does *not* do: it never takes a lock a reader holds. Readers run against a snapshot number and see a consistent view; writers advance the clock and append. That decoupling is the concurrency wedge, and the next chapter — MVCC — is how it stays correct.

4.6 The Sorted-Part Key Index

This is the hardest and most consequential piece of the storage engine: how a row is located by key without paying for a key→location map. The answer — following Apache Doris / StarRocks — is that **the sorted key column *is* the index.**

4.7 The idea

A sealed part is written **sorted by its primary key**. Therefore a row's *ordinal position* in the part is its *offset*: to find where key `k` lives, binary-search the sorted key column. There is no separate structure mapping keys to row numbers, so the resident index cost is not the ~12 bytes/row an explicit map would need — it is the Bloom filter plus min/max bounds, about **1.25 bytes/row, flat with table size.**

```
flowchart LR
    K["lookup key = k"] --> B{"min ≤ k ≤ max?<br/>(part bounds)"}
    B -->|no| SKIP["skip part"]:::x
    B -->|yes| BL{"Bloom: might contain k?"}
    BL -->|no| SKIP
    BL -->|yes| BS["binary-search the<br/>sorted key column"]:::ok
    BS --> HIT["ordinal → row"]
    classDef x fill:#f5d6d6; classDef ok fill:#d6f5d6;
```

4.8 The lookup funnel

A point lookup consults a cascade of cheap filters before it ever binary-searches, newest tier first (a later write must win):

ALGORITHM 1 — Point lookup by key

```
Input:  key k; snapshot S
Output: the visible row for k, or NONE
1  if L0 has a version of k visible to S:            ▷ newest writes first
2      return that version
3  for each sealed part P, newest to oldest:
4      if k < P.min_key or k > P.max_key: continue    ▷ ALG 11: bounds skip
5      if not P.bloom.might_contain(k):    continue    ▷ ALG 15: Bloom skip
6      o ← BinarySearchKey(P, k)                       ▷ ALGORITHM 2
7      if o found and version at o is visible to S:
8          return the row at ordinal o
9  return NONE
```

Each part that certainly lacks `k` is dropped by a **bounds comparison** (min/max) or a **Bloom probe** — neither touches the column data on disk. Only a part that *might* hold `k` is searched, and the search is over the small resident key run.

4.9 Binary search within a part

ALGORITHM 2 — Binary search the sorted key column

```
Input:  part P (sorted by key), key k
Output: an ordinal o with P.key(o) = k, or NOTFOUND
1  lo ← 0;  hi ← P.len − 1
2  while lo ≤ hi:
3      mid ← (lo + hi) / 2
4      c ← total_cmp(P.key(mid), k)                  ▷ total order over Values
5      if c < 0: lo ← mid + 1
6      elif c > 0: hi ← mid − 1
7      else: return mid                              ▷ found
8  return NOTFOUND
```

Keys are compared with `total_cmp`, a total order over all value types (integers compared exactly — never routed through `f64`, which would conflate integers beyond 2^3 and corrupt an integer key). For a duplicate-tolerant scan the search extends left/right from `mid` to the run of equal keys.

4.10 Any-type keys, and the hidden rowid

The key column may be any type — integer, text, float, boolean, date, decimal — because `total_cmp` orders them all. A table with **no** declared key gets a hidden auto-increment `_rowid`; it is still a single sorted key column, just invisible to `SELECT *`. So "keyless" costs nothing extra: there is no second index for the rowid — the sorted rowid column *is* the index, exactly as for a user key.

4.11 Why this shape

Proposition 1 (Index cost is flat). The resident per-row index cost of the sorted-part scheme is independent of the number of rows.

Proof sketch. The resident structures are the Bloom filter (a fixed bits-per-key budget), the per-part min/max bounds (constant per part), the per-row version stamps, and the deletion vector. None grows super-linearly with row count, and crucially there is **no** key→location map — the map is replaced by "ordinal = offset," which stores nothing. Measured at ~1.25 B/row, it stays flat as tables grow (`m0-bench`). ■

The consequence is the whole storage strategy: because the index is nearly free and flat, ChakraDB can hold row *data* on disk and keep only the index resident, and the resident index — not disk — becomes the scaling ceiling (see Limits).

4.12 The Merge / Compaction Algorithm

Writes create parts; parts accumulate; scans slow down. Compaction is the debt payment: it merges parts into fewer, larger, sorted parts, physically drops rows no live reader can see, and collapses redundant version stamps. It is **caller-driven** — there is no background thread — so it runs when asked, and only up to a safe GC horizon.

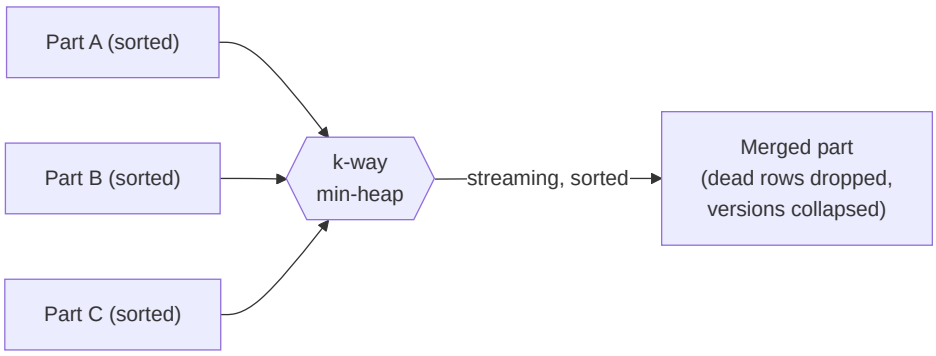
4.13 The k-way merge

Because every input part is sorted by key, merging them into one sorted part is a classic k-way merge over a heap of cursors:

ALGORITHM 8 — Merge parts

```
Input:  parts P₁..P_k (each sorted by key); reclamation horizon H
Output: one merged, sorted part
1  heap ← { (P_i.key(0), i, 0) for each non-empty P_i }   ▷ min-heap on key
2  out ← empty builder
3  while heap not empty:
4      (k, i, o) ← heap.pop_min()                          ▷ smallest current key
5      keep ← RowIsLive(P_i, o, H)                          ▷ see ALGORITHM 9
6     if keep: out.append(P_i.row(o))                     ▷ carry the surviving row
7      if o+1 < P_i.len: heap.push((P_i.key(o+1), i, o+1))  ▷ advance that cursor
8  return out.seal()                                        ▷ sorted Arrow part
```

The output is produced in sorted key order in a single streaming pass — `O(N log k)` for `N` total rows and `k` inputs — with no intermediate sort. Newer parts take precedence when keys tie, so an updated row's newest version wins.



4.14 Reclamation: which rows survive

The horizon `H` is the oldest CSN any live snapshot may observe (the GC watermark). A row can be physically dropped only when no snapshot $\geq$ `H` could still see it:

ALGORITHM 9 — Row liveness under a horizon

```
Input:  a row at ordinal o in part P; horizon H
Output: true if the row must be kept
```

```
1  if the row is not tombstoned: return true          ▷ a live version — keep
2  d ← its deletion CSN
3  if d ≤ H: return false                               ▷ deleted before/at the
4  return true                                          ▷   horizon → reclaimable
```

Line 3 is the crux: a row deleted at `d` $\leq$ `H` is invisible to *every* live snapshot (all are $\geq$ `H` >… $\geq$ `d`), so it is safe to drop. A row deleted *after* `H` is still visible to a reader on an old snapshot and **must be kept**. Getting this bound wrong is the silent-wrong-answer hazard the GC watermark chapter is devoted to.

After merging, the surviving versions' stamps are **collapsed**: if all rows of the merged part now share one creation CSN, the part stores a single `Uniform(csn)` instead of a per-row array — restoring the cheap full-part fast path for future scans.

4.15 Selection and incrementality

Two more practicalities keep compaction from being wasteful:

- **Selection.** Not every part is merged every time. A policy chooses a set of candidates (for example, similar-sized adjacent parts), so compaction does bounded work per invocation rather than rewriting the whole table.
- **Incremental persistence.** When the merged result is checkpointed, unchanged parts are skipped and a part that only *gained tombstones* since the last checkpoint has just those deletion-vector entries appended — not a full rewrite. This fixed an `O(n²)` that a naive "rewrite everything each checkpoint" would incur.

4.16 The backpressure contract

Compaction trades write amplification for scan speed. If it cannot keep up, the engine does **not** silently let scans degrade — it applies explicit ingest **backpressure** (a throttle, then a stall) keyed on the part count, surfaced in `Storage::stats()`. This is the "cost of fast writes is paid in compaction, and made visible" commitment from the cost model.

> **Proposition 6 (Merge preserves the visible database).** For any snapshot `S` $\geq$ `H`, the set of rows visible to `S` is identical before and after a merge with horizon `H`.
>
> *Proof sketch.* A merge changes physical layout, not logical content, except that it drops rows with `deleted` $\leq$ `H` (ALG 9). Such a row is invisible to any `S` $\geq$ `H` because `S` $\geq$ `H` $\geq$ `deleted` fails `S < deleted` (ALGORITHM 3). Every row visible to `S` has `deleted > H`, is therefore kept, and its `(created, deleted)` stamps are preserved (collapse only rewrites the storage form, not the values). Hence `S`'s visible set is unchanged. The requirement `S` $\geq$ `H` is exactly what the GC watermark guarantees for every live reader. ■

4.17 Zonemap Part Pruning

A selective query should not read parts that cannot contain a match. ChakraDB skips them using **zonemaps** — the per-column `(min, max)` bounds every part carries. This is DuckDB-style rowgroup pruning, and it is also the mechanism behind graph adjacency.

4.18 The idea

Each sealed part stores, per column, the minimum and maximum value over its rows. For a predicate on that column, if the part's `[min, max]` interval cannot satisfy the predicate, no row in the part can — so the whole part is skipped without touching its data.

```
flowchart LR
    Q["WHERE x >= 500"]
    P1["Part A<br/>x ∈ [1, 120]"]:::skip
    P2["Part B<br/>x ∈ [300, 900]"]:::hit
    P3["Part C<br/>x ∈ [950, 999]"]:::hit
    Q --> P1 & P2 & P3
    classDef skip fill:#f5d6d6,stroke:#c00; classDef hit fill:#d6f5d6,stroke:#0a0;
```

Part A (`max = 120 < 500`) is skipped; B and C are scanned.

4.19 The exclusion test

The predicate is compiled to a conservative *excludes* test: given a part's bounds, can this predicate be satisfied by *no* row in the part?

ALGORITHM 11 — Predicate excludes a part

```
Input:  predicate φ; part bounds (per column) [min_c, max_c]
Output: true if NO row in the part can satisfy φ (safe to skip)
1  match φ:
2    (col c) = v :  return v < min_c  or  v > max_c        ▷ value out of range
3    (col c) < v :  return min_c ≥ v                        ▷ all values ≥ v
4    (col c) ≤ v :  return min_c > v
5    (col c) > v :  return max_c ≤ v                        ▷ all values ≤ v
6    (col c) ≥ v :  return max_c < v
7    φ₁ AND φ₂ :  return excludes(φ₁) or excludes(φ₂)       ▷ either kills it
8    φ₁ OR  φ₂ :  return excludes(φ₁) and excludes(φ₂)      ▷ both must kill it
9    otherwise :  return false                              ▷ unsure ⇒ keep (safe)
```

The test is **conservative**: it returns `true` only when it can *prove* the part holds no match, and `false` (keep the part) whenever it is unsure. So pruning never changes an answer — it only avoids work.

The scan then drops every fully-materialised part the predicate excludes:

```
segments ← scan_segments(snapshot)
segments.retain(seg → not (seg is a sealed part P and excludes(φ, P.bounds)))
```

4.20 Two ways it is used

As a SQL accelerator. A selective `WHERE x = k` or `WHERE x BETWEEN a AND b` prunes to the parts whose bounds overlap — the interpreter's second-stage router sends such queries here rather than to a full vectorized scan (see Query Routing).

As a graph adjacency index. A graph edge key encodes `(src, dst)` src-major, so "neighbors of X" is a key-range scan `key ∈ [(X,0), (X+1,0))`. Zonemap pruning on the key column touches only the parts holding `src = X`. This is clustered adjacency — the graph traversal primitive falls directly out of ALGORITHM 11.

4.21 The scaling property

Proposition 7 (O(1) selective range scans). For a key-range query over a table sorted by that key, the number of parts scanned depends on the range's selectivity, not the table size.

Proof sketch. Parts are sorted by key, so each part's key-bounds interval is disjoint and ordered. A key range `[lo, hi)` overlaps only the contiguous run of parts whose intervals intersect it — a count that grows with `hi - lo`, not with the number of parts. Every other part is excluded by ALG 11 (lines 2–6). Measured: a needle range scan stays ~1 ms as the table grows 100× (ClickBench Q13, Part IX). ■

4.22 The correctness guarantee, restated

Pruning removes only parts it *proves* empty of matches, so the produced answer is identical to a full scan. The unit tests exercise the boundaries directly — equality at the min/max edges, swapped operands (`v = col`), `AND`/`OR` combinations, and missing bounds — and an end-to-end suite scans a multi-part table to confirm a pruned range returns exactly the matching rows.

4.23 Bloom Filters & the Lookup Funnel

A point lookup should not read a part that does not contain the key. The min/max bounds catch some parts; a **Bloom filter** catches the rest — it answers "could key `k` be here?" from a few bits in memory, with no disk touch and no false negatives.

4.24 What a Bloom filter guarantees

A Bloom filter is a bit array with `h` hash functions. To *insert* `k`, set the `h` bits it hashes to; to *test* `k`, check those `h` bits.

- If any bit is `0`, `k` is **definitely absent** — a true negative.
- If all `h` bits are `1`, `k` is **probably present** — possibly a false positive.

Crucially there are **no false negatives**: a key that was inserted always tests positive. That is exactly the property a lookup filter needs — it may occasionally fail to skip a part, but it never wrongly skips one that holds the key.

ALGORITHM 15 — Bloom membership test

```
Input:  key k; part Bloom filter B with h hash functions
Output: DEFINITELY_ABSENT, or MAYBE_PRESENT
1  seed ← value_seed(k)                              ▷ reduce any Value type to 64
 ↪  bits
```

```
2  for i in 0..h:
3      bit ← hash(seed, i) mod |B|
4      if B[bit] = 0: return DEFINITELY_ABSENT        ▷ a single 0 bit settles it
5  return MAYBE_PRESENT
```

`value_seed` maps a key of any type to a 64-bit seed: an integer maps to itself (so the integer path is unchanged and collision-free in that dimension), and text, float, boolean, and decimal keys get a deterministic reduction.

4.25 Its place in the funnel

The Bloom test is the second filter in the point-lookup funnel, after the min/max bounds and before the binary search:

```
flowchart LR
    K["lookup k"] --> BND{"min ≤ k ≤ max?"}
    BND -->|no| S1["skip"]:::x
    BND -->|yes| BLM{"Bloom: maybe present?"}
    BLM -->|absent| S2["skip"]:::x
    BLM -->|maybe| BS["binary search<br/>(reads the key run)"]:::ok
    classDef x fill:#f5d6d6; classDef ok fill:#d6f5d6;
```

The ordering is deliberate — cheapest test first. The bounds check is two comparisons; the Bloom probe is a handful of bit reads; only a part that passes both pays for a binary search over its resident key run. A part that certainly lacks the key is dropped before any of its data is examined.

4.26 Sizing and cost

The filter trades a small, fixed number of bits per key for a low false-positive rate. In ChakraDB it is part of the ~1.25 B/row resident index budget (Proposition 1): the Bloom bits, the min/max bounds, the version stamps, and the deletion vector together stay flat with table size, which is what lets the engine keep data on disk and the index in memory.

> **Proposition 11 (No false negatives lookups are correct).** The Bloom skip in ALGORITHM 15 never causes a lookup to miss a key that is present.
>
> *Proof sketch.* If key `k` is in the part, it was inserted, so all `h` of its bits are `1`, so ALG 15 returns `MAYBE_PRESENT` and the funnel proceeds to the binary search that finds it. The filter can only err the other way — a `MAYBE_PRESENT` for an absent key — which costs a wasted binary search, not a wrong answer. This is asserted directly: over many keys, the filter has zero false negatives (`bloom` tests). ■

4.27 Why it matters for concurrency

Point lookups are the transactional read shape, and they run on the interpreter precisely because this funnel makes them `O(log n)` with almost no I/O. That keeps the transactional path fast and lock-free while analytical scans run on the same snapshot — the HTAP split the router exploits.

Chapter 5

MVCC & Snapshot Isolation

Time is what keeps everything from happening at once.
— Ray Cummings

Multi-Version Concurrency Control is the reason readers never block writers. This chapter is the *model*; the visibility algorithm and the GC watermark get the full treatment in Part III.

5.1 The commit sequence number (CSN)

There is one piece of global state every writer touches: a monotonic 64-bit counter, the **commit sequence number**. Allocating a CSN is a single atomic increment. A **snapshot** is just a CSN — a number naming an instant.

Every row version carries two stamps:

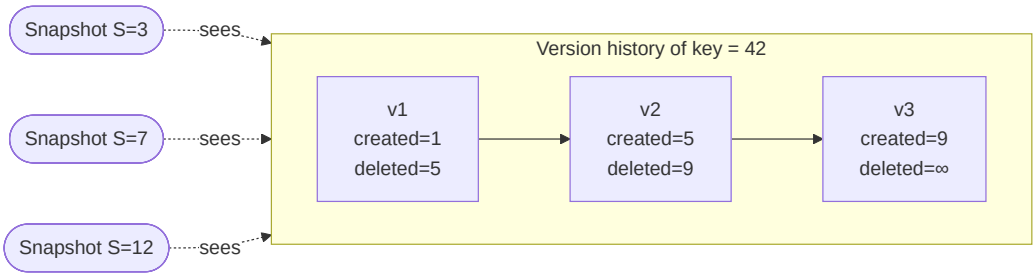
- `created` — the CSN at which this version came to exist.
- `deleted` — the CSN at which it was superseded or removed (or `NEVER_DELETED`).

5.2 The visibility predicate

A version is visible to a snapshot `S` if and only if it was created at or before `S` and not yet deleted as of `S`:

$$\texttt{visible(S, created, deleted)} \iff \texttt{created} \le \texttt{S} < \texttt{deleted}$$

That is the whole rule. It is a pure function of three integers — no locks, no undo log, no read view to construct.



At `S=3`, only `v1` satisfies `created` $\leq$ `3` `<` `deleted` ($1 \leq 3 < 5$). At `S=7`, only `v2` ($5 \leq 7 < 9$). Exactly one version of a live key is visible to any snapshot — which is what makes an `UPDATE` a create-plus-delete rather than a mutation.

5.3 Why cold data is free

The predicate has two fast-path corollaries that make ChakraDB's scans cheap:

A part created entirely at or before `S` and with no deletions after `S` is "fully

visible" — every one of its rows passes, so the scan skips the per-row check entirely.

Concretely, a part stores a uniform `created` stamp and the minimum `deleted` CSN across its rows. If `created_max` $\leq$ `S` and `S < min_deleted`, the whole part is visible with a single comparison. A cold, unmodified part — the vast majority of a large table — pays **zero** per-row version cost on a scan. Only recently-modified parts and the L0 buffer pay the per-row predicate. This is the "cost of concurrency is paid only by data that was recently modified" principle, from Neumann et al.'s MVCC design, made concrete.

5.4 Snapshot isolation, across tables

The CSN clock is global to a database, so a snapshot is consistent **across every table at once.** An application reading two tables at snapshot `S` observes both as of the same instant — no torn cross-table view. This is what lets the graph layer build a consistent adjacency from an edges table while a nodes table is also being written.

5.5 Readers do not block writers — and the reverse

- A **reader** takes a snapshot number and scans the parts visible to it. It grabs the part list under a brief read lock, then scans **lock-free**; the parts are immutable, so nothing a writer does can change what the reader is reading.
- A **writer** advances the clock and appends a new version. It never waits on a reader.

The one serialization point is *per table*: two writers to the *same* table take a short write lock for the L0 insert. Different tables write concurrently. (This is the deliberate trade in the cost model.)

5.6 Garbage collection needs a watermark

Because old versions stay until compaction reclaims them, compaction must not drop a version some live snapshot can still see. ChakraDB tracks a **live-snapshot registry**: a long-running reader (a query or a transaction) *pins* its snapshot, and compaction's reclamation horizon is held back to the oldest pinned snapshot. A reader on an old snapshot keeps its view even as newer writes are compacted away. The full mechanism — and why it is correct under concurrency — is in The GC Watermark.

5.7 Durability of the clock

The CSN is monotonic and never reset. On recovery, the counter is raised above the highest CSN any replayed record used, so a recovered database never reissues a stamp — two rows can never collide on a version number. The counter is 64-bit: $\sim 1.8 \times 10^{1}$ total mutations over a database's entire lifetime.

5.8 MVCC Visibility

Visibility is the pure function at the heart of snapshot isolation. It decides, for a snapshot `S` and a row version stamped `(created, deleted)`, whether that version is the one `S` should see. Everything

about ChakraDB's non-blocking reads follows from how cheap this function is.

5.9 The predicate

ALGORITHM 3 — Version visibility

```
Input:  snapshot S (a CSN); version stamps created, deleted
Output: true if the version is visible to S
1  return created ≤ S  and  S < deleted          ▷ deleted = ∞ if never deleted
```

That is the whole rule: a version is visible iff it was created at or before `S` and not yet deleted as of `S`. It is three integer comparisons — no lock, no undo log, no read-view to materialize.

An `UPDATE` is modeled as *delete-old + create-new*: the old version's `deleted` is set to the new CSN, and the new version's `created` is that same CSN.

Proposition 2 (Exactly one live version). For any live key and any snapshot `S`, exactly one version satisfies `created ≤ S < deleted`.

Proof sketch. The versions of a key form a chain: `v₁(c₁, d₁), v₂(c₂, d₂), …` with `d_i = c_{i+1}` (each delete coincides with the next create), the last version having `deleted = ∞`. The intervals `[c_i, d_i)` therefore **partition** the CSN line from `c₁` to `∞`. Any `S ≥ c₁` lands in exactly one interval; any `S < c₁` lands in none (the key did not exist yet). ■

```mermaid
flowchart LR
    subgraph chain["Version chain of one key — intervals partition the CSN line"]
      direction LR
      A["[1, 5)"] --> B["[5, 9)"] --> C["[9, ∞)"]
    end
    S3(["S=3 → [1,5)"]):::a --> A
    S7(["S=7 → [5,9)"]):::b --> B
    S12(["S=12 → [9,∞)"]):::c --> C
    classDef a fill:#bde0fe; classDef b fill:#a2d2ff; classDef c fill:#8ecae6;
```

5.10 Why cold data is free

The predicate has a batched fast path that is the reason a large scan is cheap. Instead of testing every row, a scan tests the *whole part* first:

ALGORITHM 4 — Full-part visibility fast path

```
Input:  snapshot S; part P with uniform created stamp c_max
        and minimum deletion m_del over its rows
Output: how to scan P under S
1  if c_max ≤ S and S < m_del:                  ▷ every row created ≤ S,
2      scan ALL rows of P — no per-row check    ▷ none deleted ≤ S
3  elif S < c_min:                              ▷ part entirely in the future
4      skip P                                    ▷ nothing visible
5  else:
```

```
6       for each row r in P:                              ▷ the slow path
7           if ALGORITHM 3 holds and r not tombstoned: emit r
```

Line 1 is the corollary that matters: a part whose newest creation is $\leq$ `S` and whose earliest deletion is `> S` is **fully visible** — every row passes, so the scan emits the batch with a single comparison and *zero* per-row work.

> **Proposition 3 (Cold-scan cost).** A scan at `S` pays the per-row visibility check only on parts modified in the window the snapshot straddles; cold, unmodified parts pay `O(1)` per part.
>
> *Proof sketch.* A cold part has a uniform `created` $\leq$ `S` (it was written long ago) and no deletions after `S`, so it takes the fast path (ALG 4, line 1) — one comparison for the entire part. Only parts with a `created` or a `deleted` inside `(S_min, S_max]` fall to the slow path. Hence a billion-row table with a thousand recent mutations checks a thousand rows, not a billion — the "cost of concurrency is paid only by recently-modified data" principle, from Neumann et al. ■

5.11 Where the stamps live

Per-part, ChakraDB stores the version stamps compactly: a part whose rows all share one CSN stores a single `Uniform(csn)` rather than a stamp per row (the common case for a bulk-sealed part), and a **deletion vector** records which ordinals are tombstoned and at which CSN. So the fast path's `c_max` and `m_del` are `O(1)` to read, and the slow path consults the deletion vector rather than rewriting the part.

5.12 The consequence for concurrency

Because visibility is a pure function of a snapshot number and immutable per-row stamps, a reader needs **no lock**: it fixes `S`, grabs the (immutable) part list, and scans. A concurrent writer advances the clock and appends new versions with higher CSNs — invisible to `S`. This is the mechanism behind "readers never block writers," and it is why the same snapshot feeds analytics, transactions, and the graph CSR consistently.

5.13 The GC Watermark & Live-Snapshot Registry

Compaction reclaims space by dropping row versions no one can see any more. Get the "no one can see" test wrong and you get the worst kind of bug: a silent wrong answer for a reader on an old snapshot. This chapter is how ChakraDB gets it right — the correctness guarantee behind the concurrency wedge.

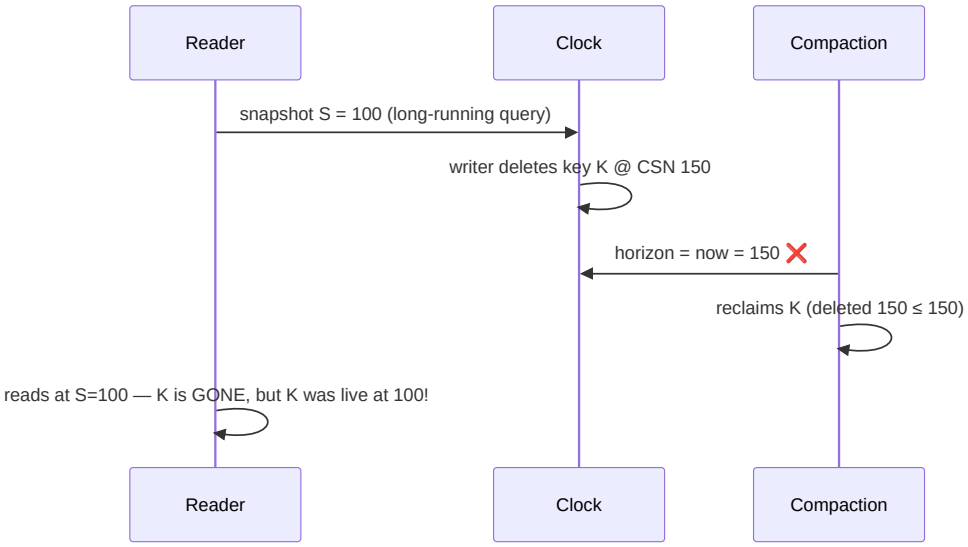
5.14 The hazard

Compaction, when it merges parts, physically reclaims rows whose deletion CSN is at or before a **horizon**. The rule is:

> The horizon must be $\leq$ **the oldest CSN any live snapshot may still observe.** Reclaim anything newer, and a reader holding an older snapshot loses a row it should

still see.

The danger case, if the horizon were just “now”:



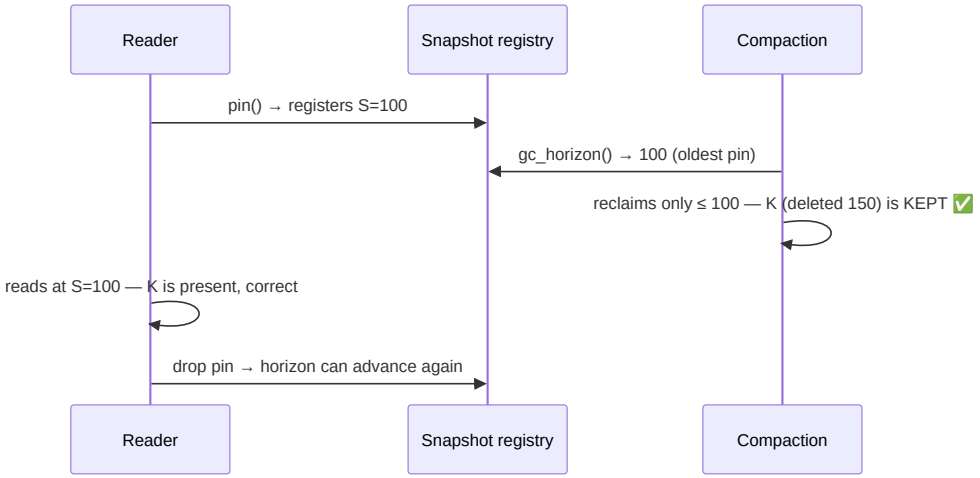
At `S=100`, key `K` (deleted at 150) *should* be visible. A horizon of “now” would have reclaimed it.

5.15 The fix: a live-snapshot registry

ChakraDB tracks the oldest live reader and holds the horizon back to it. The CSN generator keeps a small registry — a map from CSN to a count of live *pins*:

- A read that must outlive a clock advance — a query or a transaction — **pins** its snapshot: it registers its CSN and gets an RAII guard.
- **`gc_horizon()`** = the smallest pinned CSN, or the current clock if none are pinned.
- Compaction reclaims only up to `gc_horizon()`.

```
pin():                        gc_horizon():               on guard drop:
  lock registry                 lock registry               lock registry
  csn = current()               h = min(registry keys)      registry[csn] -= 1
  registry[csn] += 1               or current() if empty     if 0: remove csn
  return guard(csn)             return h                    unlock
```



5.16 The subtle part: no window

The correctness hinges on one detail: **reading the current CSN and registering the pin happen under the same lock that `gc_horizon` uses.** Otherwise a compaction could compute a horizon *above* a pin that is about to register at a lower CSN — and the race is back.

Because both operations are linearized by the one registry lock, a compaction that runs *after* a pin registers sees that pin (so its horizon $\leq$ the pin), and a compaction that runs *before* the pin

registers used a horizon $\leq$ the pin's snapshot anyway (the clock only advances). Either way the horizon never exceeds a live snapshot. The proof is small precisely because the critical section is small.

5.17 What pins, and what doesn't

Only reads that could **outlive a clock advance** need to pin:

- The **SQL interpreter** pins for the duration of a `SELECT`/`UPDATE`/`DELETE`.
- The **DataFusion bridge** pins for the whole query.
- A **transaction** pins from `BEGIN` until commit/rollback.
- A **`GraphView`** pins while it builds its CSR.

A transient read at `current` needs no pin — nothing it can see is reclaimable, because the horizon can never exceed `current`. So the hot path stays lock-free; the registry is touched once per long read, not per row.

5.18 Why compaction is safe to be lazy

Compaction is **caller-driven** — there is no background thread. Combined with the registry, that means space is reclaimed only when you ask, and only up to the oldest live reader. `Storage::compact_all()` uses `gc_horizon()` automatically; the low-level `Database::compact_all(horizon)` takes an explicit horizon for callers who know a safe one.

5.19 Regression-tested

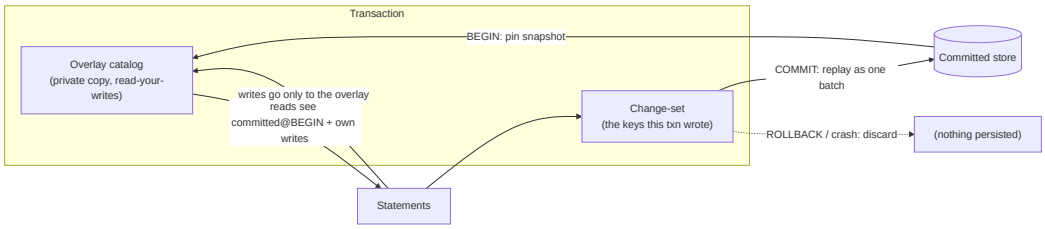
The guarantee is pinned down by a test: a reader pins a snapshot, a writer deletes a key at a later CSN, compaction runs — and the pinned reader **still sees the row**; after the pin drops, the row becomes reclaimable. The test also checks the horizon tracks the oldest of several concurrent pins (`tests/gc_watermark.rs`). It would fail against the naive "horizon = now," so it genuinely guards the fix.

5.20 Transactions

`BEGIN … COMMIT` gives ChakraDB ACID multi-statement transactions with **snapshot isolation** and **first-committer-wins** conflict detection. The design is a private **overlay** that is invisible until commit, so a crash or `ROLLBACK` simply discards it.

5.21 The overlay model

A transaction is a private catalog initialized from a committed snapshot, plus a change-set to replay on commit:



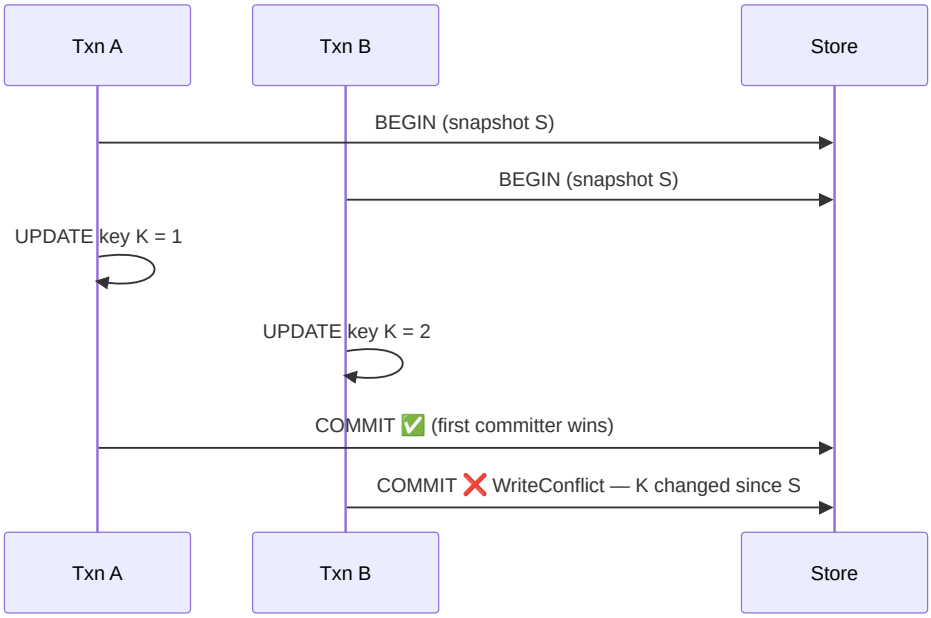
- **Reads** inside the transaction see the committed state as of `BEGIN` plus the transaction's own writes (read-your-writes), and *nothing* uncommitted from other connections.
- **Writes** go only into the overlay and the change-set — never the real store or the WAL — so rolling back is free and a crash loses nothing.
- **COMMIT** replays the change-set to the real store, which logs it as **one** crash-atomic WAL record.

5.22 Snapshot isolation

The transaction pins its `BEGIN` snapshot (which also holds the GC watermark back, so the versions it may read are not compacted away). Every statement in the transaction reads that one consistent instant, extended by its own writes. Other connections' commits are invisible.

5.23 First-committer-wins

At `COMMIT`, ChakraDB checks whether any key the transaction wrote was changed by another committed transaction since `BEGIN` — i.e. whether the key's current committed value differs from what the transaction saw at `BEGIN`. If so, the commit fails with a **write conflict** and the transaction is aborted; retry it.



Synthetic-rowid inserts never conflict — each is a brand-new row with a fresh key.

5.24 Crash-atomic commit

Because the whole change-set is handed to the durable backend as one batch, it is logged as a single `WalRecord::Txn`. A WAL record is framed with a length and CRC, so recovery applies it **all or nothing**: a crash mid-commit leaves a torn frame that is discarded, and the transaction never partially appears. This is verified by truncating the log at every byte and asserting the row count

is only ever "baseline" or "baseline + the whole transaction."

5.25 Scope

Statements in a transaction run on the **single-table interpreter** (the overlay is materialized from the committed snapshot on first touch). Joins and subqueries belong *outside* a transaction — use them on the autocommit path where DataFusion is available. DDL (`CREATE TABLE`) inside a transaction is applied immediately and is not rolled back in v1.

5.26 Example

```
BEGIN;
  UPDATE accounts SET balance = balance - 100 WHERE id = 1;
  UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;   -- both, atomically, or neither
```

Outside an explicit transaction, every statement autocommits.

Chapter 6

Durability: WAL, Group Commit & Recovery

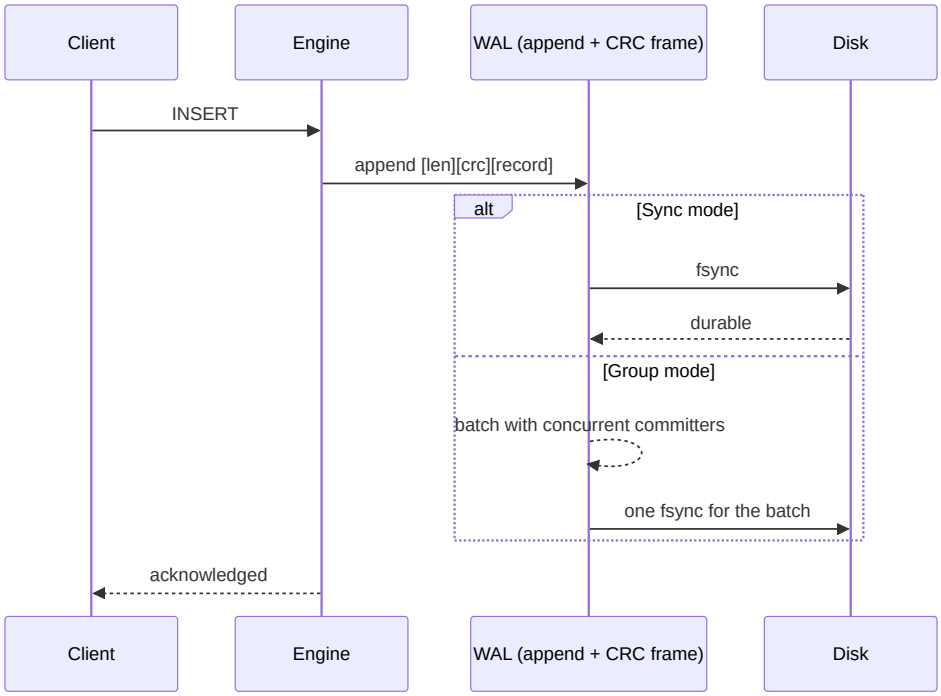
The palest ink is better than the best memory.

— *Chinese proverb*

A durable ChakraDB (`Storage`) guarantees that an acknowledged write survives a crash. The mechanism is a classic **write-ahead log** with **group commit**, a **manifest** for the catalog and part set, and a **recovery** pass that replays the log. This chapter is the shape; the WAL algorithm and recovery chapters have the byte-level detail.

6.1 Write-ahead logging

Every mutation is appended to the WAL **before** it is acknowledged. A record is framed with a length and a CRC, so a crash mid-append leaves a *torn* final frame that recovery detects by checksum and discards — the log's valid prefix is always recoverable.



6.2 Three durability modes

The trade between latency and the guarantee is a knob:

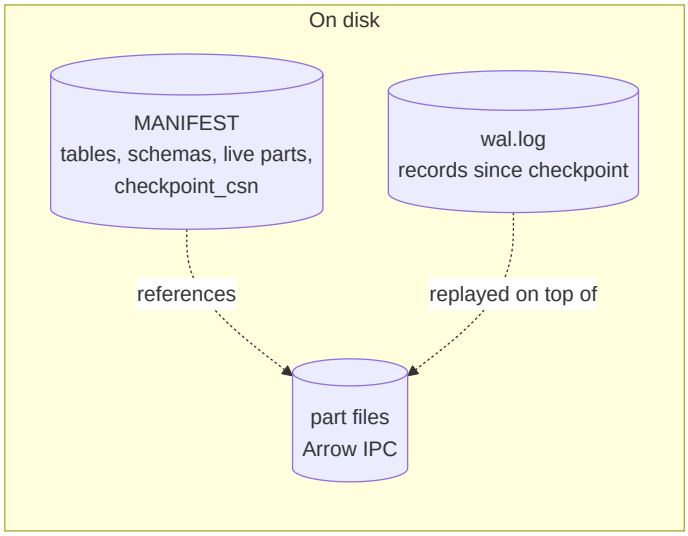
Mode	Guarantee	Cost
`Sync`	every write `fsync`'d before ack	strongest, one fsync/write

Mode	Guarantee	Cost
`Group`	concurrent writers share one `fsync`	strong, amortized
`Async`	acked before `fsync`	fastest; a crash may lose the last unflushed writes

Group commit is the default sweet spot: when many threads commit at once, their records are appended and made durable with a *single* `fsync`, so throughput scales with concurrency instead of paying a sync per write.

6.3 The manifest and checkpointing

The WAL grows without bound if never trimmed. A **checkpoint** makes the in-memory state durable in parts, records the live part set and each table's schema in the **manifest**, and advances a `checkpoint_csn` — after which the log *before* that point is reclaimable. Checkpointing is incremental: unchanged parts are skipped, and parts that only gained tombstones get just those appended. See Checkpointing.



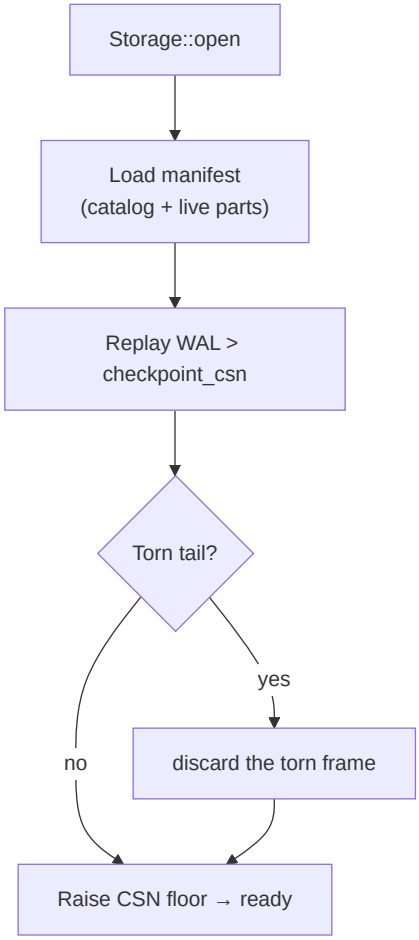
6.4 Recovery

On open, recovery reconstructs the exact acknowledged state:

1. Read the **manifest** — the catalog (tables + schemas) and the live part set as of the last checkpoint. Part *indexes* are loaded resident; part *data* is faulted lazily on first touch (fast reopen).
2. Replay the **WAL** past `checkpoint_csn`, applying each record to the in-memory tables. A

torn final frame is discarded.

3. Raise the CSN clock above the highest replayed stamp, so no version number is ever reissued.



6.5 How well it's tested

Durability is the property you cannot get wrong, so it is tested adversarially:

- **10,000 randomized crash trials** verify that every acknowledged write survives, in all three durability modes (`crash_consistency`).

- **Durable SQL** adds tens of thousands more crash trials — millions of acknowledged writes verified across integer, text, and keyless schemas (`durable_sql_crash`).
- A committed multi-statement transaction is one WAL record, so truncating the log at *every byte* leaves the transaction either fully applied or fully absent — never partial (`torn_commit_record_is_all_or_nothing`).

6.6 The single I/O seam

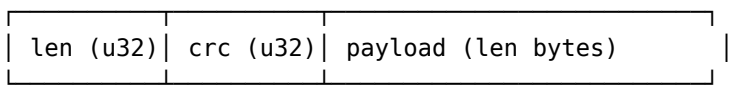
Everything reaches disk through one `trait Io` (see The I/O Abstraction) — a real POSIX backend for production and an in-memory, fault-injecting backend (`MemIo`) that makes those thousands of crash trials possible without touching a real disk. The durability logic is identical over both.

6.7 Write-Ahead Logging & Group Commit

Durability is the promise that an acknowledged write survives a crash. ChakraDB keeps it with a write-ahead log: every mutation is framed, checksummed, and made durable *before* the write is acknowledged. Group commit amortizes the `fsync` so throughput scales with concurrency.

6.8 Record framing

Each WAL record is wrapped in a self-describing frame so a torn write is detectable:



The payload is one of: `Insert{table, csn, row}`, `Delete{table, csn, key}`, `Seal{table, csn, part_id}`, `Checkpoint{csn}`, or `Txn{ops}` (a whole committed transaction as one record). The length lets recovery find the next frame; the CRC lets it reject a frame torn by a crash mid-append.

6.9 The append

ALGORITHM 5 — WAL append and acknowledge

```
Input:  record R; durability mode M ∈ {Sync, Group, Async}
Output: acknowledgement (the write is durable per M)
1  frame ← [len(R)] ++ [crc32(R)] ++ encode(R)
2  acquire the append lock                          ▷ assigns byte offsets in order
3  append frame to the log at the current end
4  release the append lock
5  case M of
6    Sync:  fsync()                                  ▷ durable before returning
7    Group: wait for the group fsync (ALGORITHM 6)   ▷ shared with concurrent
 ↪  writers
8    Async: return now                               ▷ a crash may lose the last frames
9  return OK
```

The append lock is held only for the byte-copy that assigns offsets; the `fsync` (the expensive part) happens outside it, so writers do not serialize on the disk.

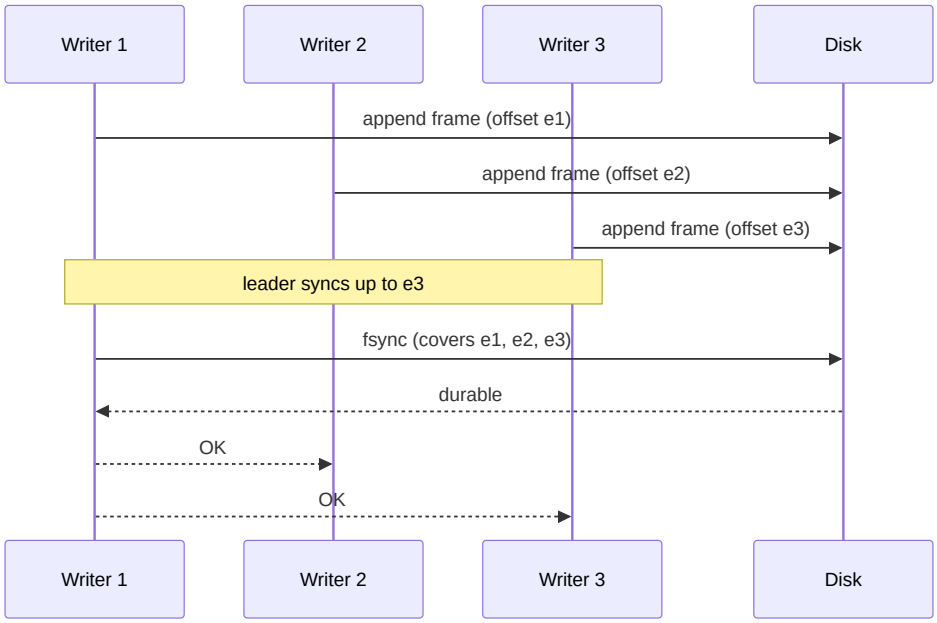
6.10 Group commit

When many threads commit at once, forcing one `fsync` per write wastes the disk's throughput — a single `fsync` makes *everything appended so far* durable. Group commit exploits that:

ALGORITHM 6 — Group commit

```
Input:  a stream of concurrent committers, each having appended its frame
Output: one fsync makes a whole batch durable
1  each committer records the log end e_i after its append, then waits
2  one committer becomes the LEADER:
3      target ← current log end                    ▷ covers every frame ≤ target
4      fsync()                                      ▷ a single disk sync
5      publish durable-through = target
6  every waiter with e_i ≤ target wakes and returns OK
```

Because an `fsync` covers all bytes written before it, `k` concurrent commits need one sync instead of `k`. The measured effect is that syncs-per-append drops sharply as concurrency rises — group commit is why durable throughput scales with writers rather than collapsing to disk-sync latency.



6.11 The three modes

Mode	Line	Guarantee	When to use
`Sync`	6	every write `fsync`'d before ack	strictest; low write rate
`Group`	7	concurrent writers share one `fsync`	default; high concurrency

Mode	Line	Guarantee	When to use
`Async`	8	acked before `fsync`	fastest; a crash may lose the last unflushed writes

6.12 The crash-atomicity of a transaction

A committed transaction is written as **one** `Txn` record — a single frame. Since a frame is all-or-nothing under the CRC, recovery applies the whole transaction or none of it.

> **Proposition 4 (Commit atomicity).** A crash during the commit of a transaction leaves the database in a state with either all of the transaction's writes or none.
>
> *Proof sketch.* The transaction's writes are one framed record. If the crash occurs before the frame's bytes (and their covering `fsync`) are durable, recovery sees no valid frame there (the CRC fails on the torn tail) and discards it — *none*. If the frame and its `fsync` completed, recovery replays the whole frame — *all*. There is no third outcome because the frame is the unit of durability. This is verified empirically by truncating the log at **every** byte and asserting the recovered row count is only ever the pre-transaction value or the post-transaction value (`torn_commit_record_is_all_or_nothing`). ■

Recovery — how the log is replayed to reconstruct the acknowledged state — is the next chapter.

6.13 Crash Recovery

Recovery reconstructs the exact acknowledged state after a crash: the catalog from the manifest, the data from the parts, and everything since the last checkpoint by replaying the WAL — discarding any torn tail.

6.14 The procedure

ALGORITHM 7 — Recovery

```
Input:  a database directory (manifest, part files, wal.log)
Output: an open database at the last acknowledged state
1  M ← read manifest                                  ▷ catalog + live parts +
 ↪  checkpoint_csn
2  for each table T in M:
3      rebuild T's schema; register its live parts    ▷ indexes resident, data lazy
4  max ← M.checkpoint_csn
5  for each frame F in wal.log, in order:
6      if not crc_ok(F): break                         ▷ torn tail — stop cleanly
 ↪  (ALG 7a)
7      if F.csn ≤ M.checkpoint_csn: continue           ▷ already durable in parts
8      apply F to the in-memory tables                 ▷ Insert/Delete/Txn; skip
 ↪  unknown tables
9      max ← builtin_max(max, F.csn)
```

```
10 set the CSN clock floor to max + 1                ▷ never reissue a stamp
11 return the open database
```

Three things make this correct and fast.

Only the tail is replayed. A checkpoint has already written everything at or below `checkpoint_csn` into parts (recorded in the manifest), so recovery skips those frames (line 7) and replays only what came after — usually a small tail.

Parts load lazily. Line 3 registers each part's *index* (Bloom, bounds, version stamps, deletion vector) resident, but leaves its *column data* on disk until first touched. Reopening a large database is therefore near-instant — flat in the number of parts, not their bytes.

Unknown tables are skipped. If a frame references a table not in the manifest (it was dropped, or a `TRUNCATE` gave it a fresh id), line 8 ignores it. This is what makes `DROP TABLE` and `TRUNCATE` durable without a special log record — see the SQL surface.

6.15 The torn tail

A crash mid-append leaves a partial final frame. Recovery detects it by checksum and stops — the log's valid prefix is always recoverable:

ALGORITHM 7a — Torn-tail detection (line 6, expanded)

```
Input:  the byte at the current read position in the log
Output: the next valid frame, or STOP
1  if fewer than 8 bytes remain: STOP                ▷ no room for len+crc
2  len ← read u32;  crc ← read u32
3  if fewer than len bytes remain: STOP              ▷ payload truncated
4  payload ← read len bytes
5  if crc32(payload) ≠ crc: STOP                      ▷ torn or corrupt frame
6  return decode(payload)
```

Everything after the first bad frame is discarded — those writes were never acknowledged (their covering `fsync` had not completed), so dropping them is correct.

```
flowchart LR
    subgraph log["wal.log after a crash"]
      F1["frame 1 ✓"] --> F2["frame 2 ✓"] --> F3["frame 3 ✓"] --> T["torn frame ✗"]
    end
    F1 & F2 & F3 -->|replayed| DB[("recovered state")]
    T -.->|discarded (crc fails)| X["(dropped)"]
    style T fill:#f5d6d6
```

6.16 Why it recovers the exact acknowledged state

Proposition 5 (Recovery completeness). After recovery, the database contains every acknowledged write and no unacknowledged one.

Proof sketch. An *acknowledged* write is, by the durability mode (ALGORITHM 5), one whose frame and covering `fsync` completed — so its frame is intact and precedes the torn

tail, and it is replayed (or already in a part below the checkpoint). An *unacknowledged* write either never reached the log or sits in the torn tail; the CRC rejects the latter, so it is dropped. Hence the recovered set is exactly the acknowledged set. This is stress-tested by tens of thousands of randomized crash trials across all durability modes and schemas (`crash_consistency`, `durable_sql_crash`). ■

6.17 Clock safety

Line 10 raises the CSN clock above the highest replayed stamp. Without it, a recovered database could hand out a CSN a replayed row already used, and two rows would collide on a version number — corrupting visibility. The floor makes recovered CSNs safe: the next allocation is strictly greater than anything on disk.

Chapter 7

The Query Layer: The HTAP Router

If the only tool you have is a hammer, it is tempting to treat everything as if it were a nail.

— *Abraham Maslow*

ChakraDB runs **two** execution engines and routes each statement to whichever fits. This is the concrete shape of "buy execution, build storage": DataFusion is bought for vectorized analytics; a compact interpreter is built for the transactional and point-query path it wins.

7.1 Why two engines

Different query shapes have opposite ideal engines:

- A **key point-lookup** (`WHERE id = 42`) wants an index funnel — a Bloom probe and a binary search — not a vectorized scan operator. The interpreter wins by orders of magnitude.
- A **`COUNT(*)`** or **bare `MIN`/`MAX`** wants to be answered from metadata (row counts, zonemaps) with *no scan at all*. The interpreter does exactly that.
- A **large `GROUP BY` / join / window** wants vectorized, spill-capable operators. DataFusion wins.
- A **write** (`INSERT`/`UPDATE`/`DELETE`/`COPY`) must own the WAL and the snapshot clock. Only the interpreter writes.

Routing to a single engine would lose one side of this. So ChakraDB has a **cost-based router** that inspects the planned statement.

7.2 The routing decision

The first stage asks whether the interpreter can even plan the statement; joins, subqueries, and window functions it cannot, so they go straight to DataFusion. For the rest, a second cost stage checks the cheap-shape shortcuts — metadata answers, point lookups, and highly selective ranges that zonemap pruning makes cheaper on the interpreter than a full vectorized scan — and everything else takes the vectorized path.

7.3 The interpreter half

A hand-written, allocation-conscious evaluator over the sorted parts. It:

- resolves columns to indices at plan time (no per-row name lookup),
- evaluates predicates and projections columnar, over the Arrow batches in place,
- answers `COUNT(*)`/`MIN`/`MAX` from part metadata,
- funnels point lookups through bounds → Bloom → binary search,
- prunes parts on selective ranges,
- and is the **only** path that mutates state and writes the WAL.

It is also the **zero-heavy-dependency fallback**: build without the `datafusion` feature and the engine still runs single-table SQL on the interpreter alone.

7.4 The DataFusion half

DataFusion receives the raw SQL and executes it over an Arrow view of the current MVCC snapshot — **zero-copy**, since sealed parts already *are* Arrow record batches, handed over by reference. It brings the large SQL surface (joins, subqueries, windows, a rich function library) and vectorized, spill-capable operators. ChakraDB pins the snapshot for the query's duration, so DataFusion sees a consistent read while writers keep committing.

> **A subtlety worth stating.** Because the interpreter owns writes and DDL, a statement that fails to *plan* on the interpreter — say, a constraint violation — is a real error and is surfaced, not silently retried on DataFusion. Only a *SELECT* the interpreter cannot plan falls through to the vectorized engine.

7.5 Transactions run on the interpreter

Inside a `BEGIN … COMMIT`, statements run on the interpreter against a private overlay (read-your-writes; nothing hits the WAL until commit). Joins and subqueries belong *outside* a transaction — the transactional path is single-table by design. See Transactions.

7.6 Why this is the right split

Building a world-class vectorized engine is a multi-year effort on the axis ChakraDB *deliberately does not compete on* (raw scan speed). Buying DataFusion spends those years on storage, MVCC, durability, and graph instead — while the interpreter captures the transactional shapes DataFusion is simply the wrong tool for. The DuckDB comparison shows where each lands.

7.7 Query Routing (Cost Model)

The HTAP router decides, for each statement, whether to run it on the hand-written **interpreter** or on **DataFusion**. The decision is cheap — it inspects the *planned* statement, not the data — and it is the concrete cost model that lets one engine serve both transactional and analytical shapes well.

7.8 The two-stage decision

ALGORITHM 12 — Route a statement

```
Input:  SQL statement text
Output: the engine to execute it on
1  plan ← interpreter.plan(statement)
2  if plan failed:                                    ▷ join / subquery / window
3      if statement is a SELECT: return DataFusion    ▷ it can plan these
4      else: raise the error                          ▷ a write/DDL error is real
5  if statement is a write or DDL: return Interpreter ▷ owns WAL + snapshot clock
6  if plan is bare COUNT(*) with no filter:  return Interpreter  ▷ metadata, no
 ↪  scan
7  if plan is bare MIN(c)/MAX(c):           return Interpreter  ▷ zonemaps, no scan
8  if plan is a key point lookup:            return Interpreter  ▷ index funnel
9  if plan is a selective range that prunes ≥ 80% of rows across ≥ 4 parts:
10     return Interpreter                             ▷ pruned scan beats vectorized
11 return DataFusion                                    ▷ scans, joins, aggregation
```

The first stage (lines 1–4) is a capability check: joins, subqueries, and windows the interpreter cannot plan, so a `SELECT` using them falls through to DataFusion, while a *write* that fails to plan (e.g. a constraint violation) is a real error and is surfaced — never silently retried on the read-only engine.

The second stage (lines 5–11) is the cost model: the shapes the interpreter *wins* are captured explicitly, and everything else takes the vectorized path.

7.9 Why each branch

- **Writes / DDL (line 5).** Only the interpreter mutates state and appends to the WAL; it owns the snapshot clock.
- **Bare `COUNT(*)` (line 6).** Answered from part row-counts — no scan at all.
- **Bare `MIN`/`MAX` (line 7).** Answered from the per-part zonemaps — no scan.
- **Point lookup (line 8).** The index funnel is `O(log n)`; a vectorized scan operator would be far slower for a single row.

- **Selective range (lines 9–10).** Zonemap pruning makes the interpreter touch only a few parts; below ~20% selectivity the pruned interpreter scan beats a full vectorized scan. The threshold is a cost estimate, not a scan — it counts, from the zonemaps alone, how many rows survive pruning.
- **Everything else (line 11).** Large scans, `GROUP BY`, joins, windows, and subqueries want DataFusion's vectorized, spill-capable operators.

7.10 The estimate is metadata-only

Line 9's "prunes $\geq$ 80%" is computed without reading a row: for each part, the `excludes` test (ALGORITHM 11) plus the part's row count says how many rows survive. Summing over parts gives the surviving fraction. So the routing decision itself is `O(number of parts)`, not `O(rows)`.

Proposition 8 (Routing is answer-preserving). For any statement, the interpreter and DataFusion produce the same result; routing changes only cost.

Proof sketch. Both engines read the same MVCC snapshot and the same immutable parts. The interpreter's fast paths are exact — `COUNT(*)` from true row counts, `MIN`/`MAX` from exact zonemaps, point lookup from the sorted key, pruned scan from a conservative `excludes` that never drops a match (ALG 11). DataFusion evaluates the same relational semantics over the same Arrow data. Hence the two agree on every routed statement; the HTAP-equivalence property tests assert exactly this across randomized queries. ■

7.11 The pin

Whichever engine runs, the statement first **pins** its snapshot for the duration (so compaction cannot reclaim a version it will read — the GC watermark). Transient reads at the current clock need no pin; long reads (a full scan, a DataFusion query, a transaction) hold one.

Chapter 8

Exact Decimal Arithmetic

Round numbers are always false.

— *Samuel Johnson*

Money must not round. ChakraDB's `DECIMAL(p, s)` is exact fixed-point — an `i128` unscaled mantissa with a scale, backed by Arrow `Decimal128`, never `f64`. This chapter is how values are parsed, compared, and summed without ever touching a float.

8.1 Representation

A decimal is `(mantissa, scale)`: the number is `mantissa / 10^scale`. So `12.34` is `Decimal(1234, 2)`, and `-0.01` is `Decimal(-1, 2)`. `DECIMAL(p, s)` bounds the **precision** `p` (total significant digits, $p \leq 38$ for `i128`) and the **scale** `s`. Values are stored at the column's scale.

8.2 Exact parsing from text

The exactness starts at ingestion: a decimal literal is parsed from its **source text**, never via `f64`. `9.99` becomes `Decimal(999, 2)` directly — not the nearest double `9.9900000000000002…` rounded back.

ALGORITHM 13 — Parse and fit a decimal literal

```
Input:  the literal's text t; target type DECIMAL(p, s)
Output: Decimal(m, s), or REJECT
1  (digits, point) ← split t on '.'                  ▷ integer part, fraction
2  from ← len(fraction);  raw ← integer ++ fraction as i128   ▷ exact, base-10
3  m ← Rescale(raw, from, s)                          ▷ align to the column scale
4  if |m| ≥ 10^p: REJECT                              ▷ exceeds declared precision
5  return Decimal(m, s)
```

Line 4 enforces precision: `1000` into `DECIMAL(3,0)` (max `999`) is rejected, not silently stored — a real correctness guard for money.

Rescaling to a target scale is exact when growing, and rounds half-away-from-zero when shrinking (SQL `CAST` rounding):

```
Rescale(m, from, to):
  if to = from: return m
  if to > from: return m · 10^(to−from)              ▷ exact
  div ← 10^(from−to);  half ← div/2                   ▷ shrinking → round
  return (m + sign(m)·half) / div
```

8.3 Exact comparison

Two decimals of different scales are compared by aligning to the larger scale in `i128` — never through `f64`, which would conflate values a distributed ledger must distinguish:

```
cmp(Decimal(a, sa), Decimal(b, sb)):
  s ← max(sa, sb)
  return compare_i128( a·10^(s−sa),  b·10^(s−sb) )    ▷ exact; f64 only on overflow
```

So `10.00` and `10.0` and the integer `10` all compare equal, exactly. Decimals also order correctly against integers (widened to scale 0).

8.4 Exact aggregation and arithmetic

`SUM` of a decimal column accumulates the `i128` mantissa at the shared scale and stays exact — the interpreter's accumulator keeps a running `i128` while every value is a decimal of one scale (falling back to `f64` only on a genuine mix). `MIN`/`MAX` are exact via the comparison above. Arithmetic:

- `+` / `−` — align scales, add/subtract mantissas (checked; `f64` only on `i128` overflow).
- `×` — multiply mantissas, add scales: `Decimal(a, sa) × Decimal(b, sb) = Decimal(a·b, sa+sb)`.
- `÷`, `%` — fall back to `f64` (exact decimal division has an implementation-defined scale; documented as the one approximate operator).
- `AVG` — returns a float (a mean is inherently fractional).

Proposition 9 (Exactness of storage, comparison, and sum). For decimals that fit `DECIMAL(p, s)`, round-trip storage, comparison, and `SUM` introduce no error.

Proof sketch. Storage keeps the exact base-10 mantissa (ALG 13, parsed from text, no `f64`). Comparison aligns scales by exact `i128` multiplication. Summation adds exact `i128` mantissas at a common scale. None of these operations uses floating point, so no rounding occurs. The famous witness — `0.1 + 0.2`, which is `0.30000000000000004` in `f64` — stores and sums to exactly `0.3` (`tests/decimal.rs`). ■

8.5 Where the float boundary is

The only places a decimal meets `f64` are division/modulo, `AVG`, and an explicit cast to `FLOAT`. Everywhere a ledger cares — INSERT, comparison, `SUM`, `MIN`/`MAX`, `+`/`−`/`×` — the arithmetic is exact `i128`. That boundary is stated plainly so no one is surprised by a rounded cent.

8.6 Temporal Encoding

`DATE` and `TIMESTAMP` are **logical types over integer storage**: a date is a count of days since the Unix epoch, a timestamp a count of microseconds. That choice keeps comparison, zonemaps, keys, and MVCC working unchanged on integers, while exposing the columns to Arrow (and DataFusion) as their native temporal types.

8.7 The representation

Type	Stored as	Arrow type	Literal
`DATE`	`i64` days since 1970-01-01	`Date32`	`'YYYY-MM-DD'` or `DATE '…'`
`TIMESTAMP`	`i64` µs since the epoch	`Timestamp(µs)`	`'YYYY-MM-DD[T]HH:MM:SS[.ffffff]'`

Because the physical value is an integer, a `DATE` column sorts, prunes, and compares exactly like an integer column — a date range `WHERE d >= '2024-01-01'` prunes via zonemaps, and a date can be a primary key.

8.8 Civil ↔ days

The conversion between a calendar date and a day number uses Howard Hinnant's proleptic-Gregorian algorithm — exact, branch-light, and dependency-free (the core crate forbids `unsafe` and has no `chrono`).

ALGORITHM 14 — Days from a civil date

```
Input:  year y, month m, day d
Output: days since 1970-01-01
1  y ← y − (m ≤ 2 ? 1 : 0)                          ▷ March-based year
2  era ← (y ≥ 0 ? y : y−399) / 400
3  yoe ← y − era·400                                 ▷ year of era  [0,399]
4  doy ← (153·(m > 2 ? m−3 : m+9) + 2)/5 + d − 1     ▷ day of year  [0,365]
5  doe ← yoe·365 + yoe/4 − yoe/100 + doy             ▷ day of era   [0,146096]
6  return era·146097 + doe − 719468
```

The inverse (`days → (y, m, d)`) is the mirror image and is used for rendering. A timestamp splits into `days · 86_400_000_000 + micros_of_day`; rendering uses floored division so pre-epoch instants format correctly.

8.9 Parsing is bounded

A hostile literal must not overflow the integer math. Parsing bounds the year and uses checked arithmetic, so an out-of-range date is *rejected*, never wrapped:

```
parse_date(t):
  (y, m, d) ← split t on '-'
```

```
    if |y| > 262143 or m ∉ [1,12] or d ∉ [1,31]: REJECT   ▷ bounded → no overflow
    return days_from_civil(y, m, d)
parse_timestamp(t):
    days ← parse_date(date_part);  μs ← parse_time(time_part)
    return checked(days · 86_400_000_000 + μs)           ▷ None on overflow → REJECT
```

The bound (`|year|` $\leq$ `262143`) comfortably covers the range Arrow `Timestamp` can represent and keeps every downstream multiplication inside `i64`. Before this bound, `DATE '300000-01-01'` overflowed — a debug panic, and worse, a silent wrap to a garbage epoch in release. Now it errors cleanly.

8.10 Rendering, consistently across engines

A `DATE`/`TIMESTAMP` column stores an integer, but must *display* as a date string. Two paths render it, and they agree:

- **DataFusion** reads the native `Date32`/`Timestamp` Arrow array and renders it.
- **The interpreter** threads each projection's output type so a bare temporal column renders through the civil-from-days formatter, not as a raw integer.

So `SELECT hire_date FROM emp` shows `2024-01-15` whether the query ran on the interpreter (a point lookup) or on DataFusion (a scan) — the tests assert both paths, and a hostile literal errors instead of panicking.

8.11 Why logical-over-integer

Proposition 10 (Temporal correctness is inherited). Ordering, zonemap pruning, and MVCC visibility are correct for `DATE`/`TIMESTAMP` with no new code.

Proof sketch. Each is defined on the physical value. Dates/timestamps are stored as monotonically-increasing integers (a later instant is a larger integer), so integer ordering *is* chronological ordering; the zonemap min/max are the earliest/latest instants; and visibility compares CSNs, untouched by the column type. The only temporal-specific code is text↔integer conversion at the parse and render boundaries (ALG 14). ■

Part IV

The Graph Engine

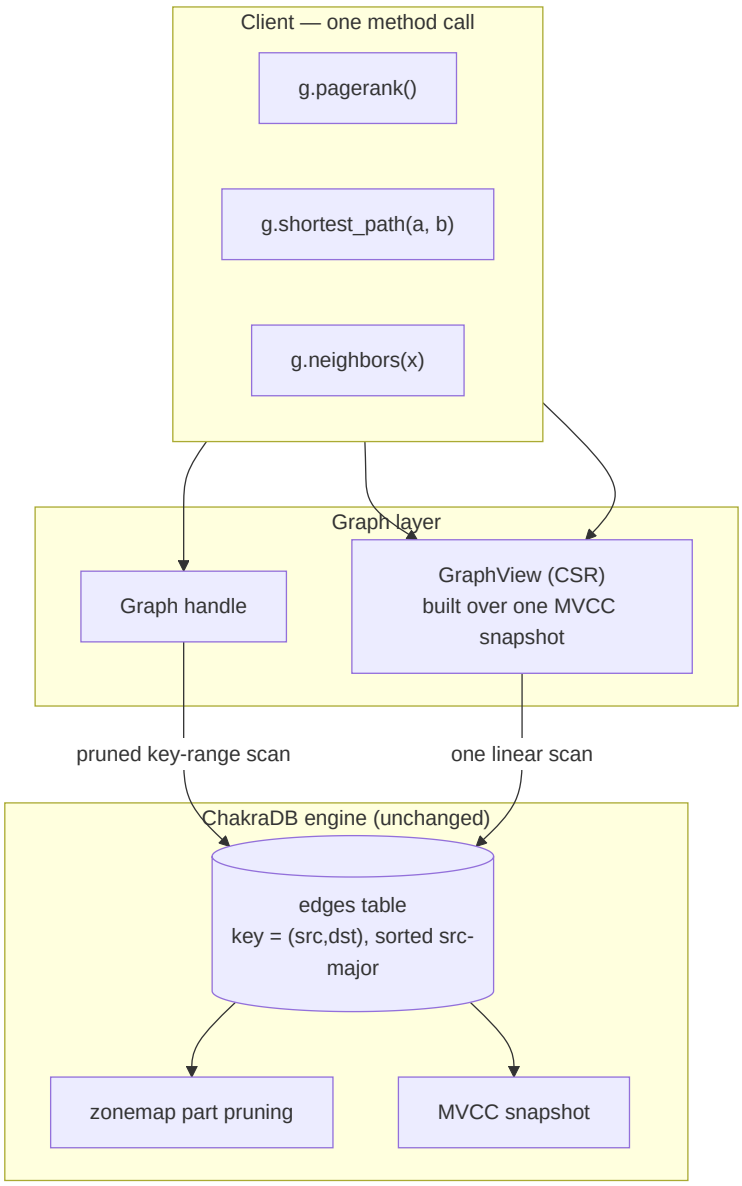
Chapter 9

Graphs on ChakraDB

Only connect!

— *E. M. Forster*

ChakraDB's graph layer is not a second database bolted onto a relational one. It is a thin layer that *exposes what the engine already is*. Three properties we built for HTAP turn out to be exactly what a graph engine wants.



9.1 The three properties

1. **Clustered adjacency, for free.** If an edge's key encodes `(src, dst)` src-major, then all of a node's out-edges are one *contiguous key range*. "Neighbors of X" becomes a key-range scan that zonemap pruning answers by touching only the parts holding `src = X`. No secondary index; the primary-key sort *is* the adjacency index. See Clustered Adjacency.

2. **Live graph analytics.** A whole-graph algorithm builds its working set from **one MVCC**

snapshot. Writers keep adding edges; the algorithm sees a consistent graph and runs to completion. This is the concurrency wedge applied to graphs — the thing pure graph libraries (a dead static copy) and lock-based graph databases cannot do in one embedded process. See Live Graph Analytics.

3. **Arrow ≈ CSR.** Edges stored sorted by `src` are already *grouped by source*, so building the CSR (Compressed Sparse Row) form every algorithm wants is a single linear, vectorized scan over Arrow columns — no sort, no hash join. See The CSR Snapshot.

9.2 What the client writes

```rust
use chakradb::{Database, Graph};
use std::sync::Arc;

let g = Graph::open(Arc::new(Database::new()), "social")?;
g.add_edge(1, 2, 1.0)?;                 // src -> dst, weight
g.add_edges([(2, 3, 1.0), (1, 3, 1.0)])?;

// Live adjacency (pruned range scan):
let nbrs = g.out_neighbors(1)?;         // [2, 3]

// Whole-graph algorithms over a consistent snapshot:
let view   = g.view()?;
let ranks  = view.pagerank(20, 0.85);   // node -> score
let path   = view.shortest_path(1, 3);  // Some([1, 3])
let comps  = view.connected_components();
let tris   = view.triangle_count();
```

The client never writes a traversal by hand and never manages a lock. The engine handles adjacency (pruned scans), consistency (snapshots), and the algorithms.

9.3 Positioning

This is the **G** in "ChakraDB — the embedded HTAP database with built-in graph capabilities." Transactions, analytics, and graph traversals run over the *same live data*, on the *same non-blocking snapshots*, in *one process*. The rest of this part builds the layer up: modeling, adjacency, CSR, and the algorithms.

> **Status.** The graph layer is real and tested (`src/graph.rs`, `tests/graph.rs`) — directed edges, adjacency, CSR views, and the core algorithm set. Node ids are currently `u32`; a native composite key (on the roadmap) will lift that and remove the key-encoding step. See Modeling a Property Graph.

9.4 Modeling a Property Graph

A property graph is **nodes and edges**, each with a label/type and properties. On ChakraDB it maps to ordinary tables — with one twist that makes traversal fast.

9.5 The tables

```
-- Nodes: keyed by node id → fast point lookup and id-range scan.
CREATE TABLE nodes (
  id     INT PRIMARY KEY,
  label  VARCHAR(64),
  props  TEXT               -- JSON, or promote hot properties to real columns
);

-- Edges: keyed (src, dst) src-major → clustered adjacency by source.
CREATE TABLE edges (
  key    INT PRIMARY KEY,   -- encode(src, dst); see Clustered Adjacency
  src    INT,
  dst    INT,
  type   VARCHAR(32),
  weight DOUBLE
);
```

The `Graph` handle manages the edge encoding for you; the schema above is what it creates under the hood (`{name}_edges`). A companion `nodes` table for node properties is optional in v1 — nodes are otherwise implied by the ids that appear in edges.

9.6 Hot vs. cold properties

Put **hot** edge properties — the ones you filter on — in real typed columns. They get zonemaps, so a query like "follow-edges created after T" prunes:

```
SELECT dst FROM edges
WHERE key >= (X<<32) AND key < ((X+1)<<32)   -- neighbors of X (clustered)
  AND type = 'follows' AND weight > 0.5;      -- pruned by zonemaps
```

Put **cold / sparse** properties in a `props` blob. This is the classic wide-vs-narrow trade: typed columns for what you query, a blob for the long tail.

9.7 Directed vs. undirected

Edges are stored **directed**. Model an undirected edge by inserting both `(u,v)` and `(v,u)`, or let the CSR builder symmetrize for undirected algorithms (connected components, triangles do this internally). For fast **backward** traversal, keep a second table keyed `(dst, src)` — the reverse adjacency.

9.8 Transactions keep it consistent

Adding an edge that touches multiple tables (an edge row, a reverse-edge row, a degree counter) belongs in **one transaction**, so a reader never sees half an edge:

```
BEGIN;
  INSERT INTO edges VALUES (encode(1,2), 1, 2, 'follows', 1.0);
```

```
  INSERT INTO edges_rev VALUES (encode(2,1), 1, 2);
COMMIT;
```

9.9 What's *not* modeled

- **Composite / multi-column keys** — not yet native, hence the `(src,dst)` encoding (the roadmap's top item).
- **Foreign keys** — an explicit non-goal; referential integrity between nodes and edges is the application's responsibility.
- **Node ids beyond 2^{31}** — the packed-key encoding's current ceiling.

The next chapters — Clustered Adjacency and The CSR Snapshot — show why this simple mapping traverses fast.

9.10 Clustered Adjacency

The single hard constraint ChakraDB puts on a graph is also the source of its adjacency speed. The engine has a **single-column primary key** and **no secondary indexes**. So "find all edges with `src = X`" is fast *only if the edge table is physically sorted by* `src`. And parts are sorted by the primary key. Therefore:

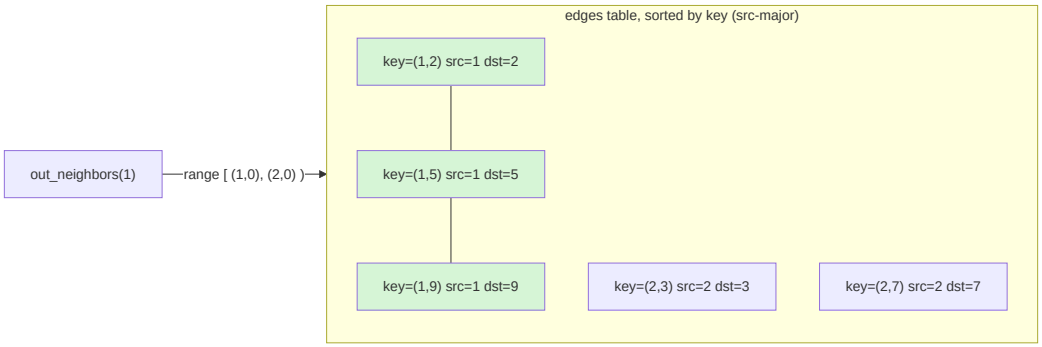
> **The edge key must sort src-major.** Encode `(src, dst)` into one sortable key, and a node's out-edges become a contiguous key range that part pruning answers cheaply.

9.11 The key encoding

The graph layer packs a directed edge into a single integer key:

```
key(src, dst) = (src << 32) | dst
```

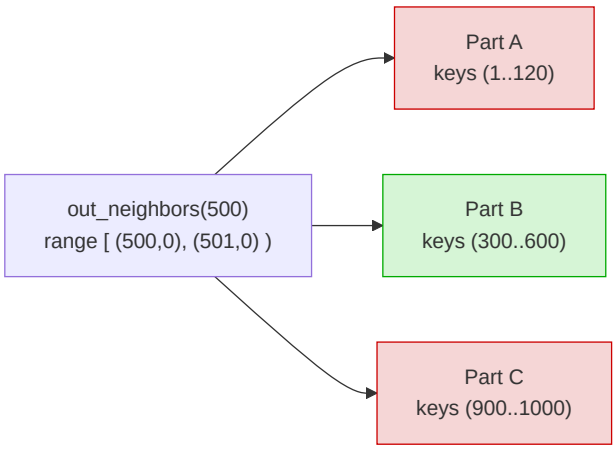
Because the high 32 bits are `src`, keys sort by `src` first, then `dst`. All of node `X`'s out-edges occupy the half-open key range `[X·2³², (X+1)·2³²)`.



`out_neighbors(1)` scans the key range `[(1,0), (2,0))`, which is exactly the green rows — contiguous, and nothing else is touched.

9.12 Why the scan is cheap: part pruning

Sealed parts are sorted by key, so each part's keys occupy a `[min_key, max_key]` interval. The neighbor scan skips any part whose interval cannot overlap the query range:



Only Part B can hold `src = 500`, so only Part B is scanned. This is the same zonemap part pruning that accelerates a selective SQL `WHERE` — reused as the graph adjacency index. In the ClickBench benchmark, a key-range scan of this shape stays effectively **O(1) as the table grows 100×** (Part IX). For a graph, that means neighbor lookup cost tracks the node's degree, not the graph's size.

The primitive in the engine is:

```
// src/table.rs — prunes parts by [min_key, max_key], scans survivors.
pub fn scan_key_range(&self, lo: &Value, hi: &Value, snap: Snapshot) -> Vec<Row>;
```

and the graph layer uses it:

```rust
pub fn out_neighbors(&self, node: NodeId) -> Result<Vec<NodeId>> {
    let lo = encode(node, 0);
    let hi = encode(node + 1, 0);        // exclusive
    // scan_key_range prunes to the parts that hold this src, over a snapshot.
}
```

9.13 Directed, and the reverse direction

Today the layer stores **directed** out-edges in one table. Two traversal patterns follow:

- **Forward** (out-neighbors): the range scan above.
- **Backward** (in-neighbors): the design keeps a second table keyed `(dst, src)`, so in-neighbors are a range scan on *that* table. (Undirected algorithms — like connected components and triangle counting — symmetrize the adjacency in the CSR; see The CSR Snapshot.)

9.14 The limit, and the fix

The packed-integer encoding caps node ids at `< 2³¹` (so keys stay positive `i64`). That is a real v1 limitation. The clean removal is a **native composite / bytes key** — letting a table declare `PRIMARY KEY (src, dst)` or a lexicographic `BYTES` key. That would make edges first-class, remove the encoding entirely, and benefit non-graph multi-column keys too. It is the highest-leverage item on the graph roadmap (see the design exploration).

Chapter 10

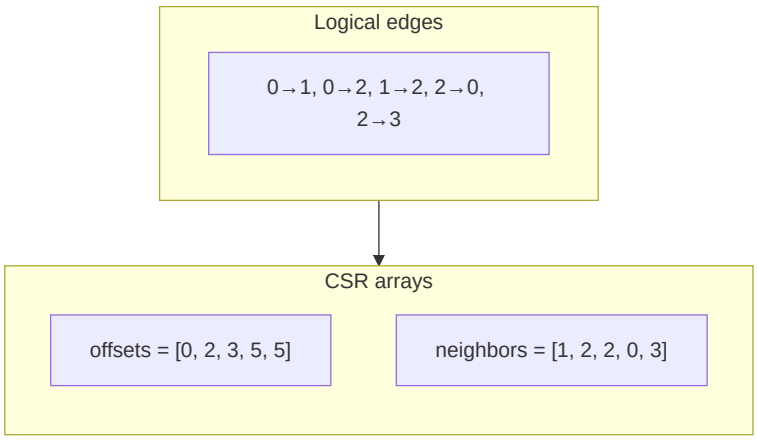
The CSR Snapshot

Simplicity is prerequisite for reliability.

— *Edsger W. Dijkstra*

Whole-graph algorithms — BFS, PageRank, connected components — do not want to issue a range scan per node. They want the **Compressed Sparse Row (CSR)** representation: a flat neighbor array plus per-node offsets, giving cache-friendly `O(1)` neighbor iteration. `Graph::view()` builds one from a single MVCC snapshot.

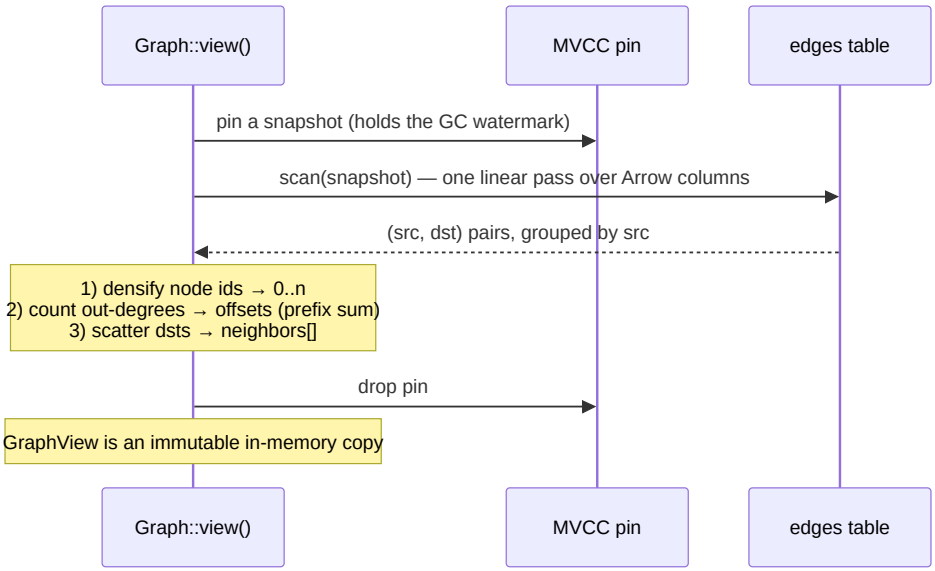
10.1 What CSR is



To iterate node `u`'s out-neighbors, take `neighbors[offsets[u] .. offsets[u+1]]`. For node 2 above: `offsets[2]=3, offsets[3]=5` → `neighbors[3..5] = [0, 3]`.

10.2 Why building it is a linear scan

The edge table is already sorted by key = `(src, dst)` src-major. So a single scan yields `(src, dst)` pairs *already grouped by source*. Building CSR is then a counting pass and a fill pass — no sort, no hash join:



The actual build (`src/graph.rs`):

```
// Dense node numbering over every id that appears.
let mut ids: Vec<NodeId> = pairs.iter().flat_map(|&(s,d)| [s,d]).collect();
```

```
ids.sort_unstable(); ids.dedup();
let index: HashMap<NodeId,u32> = /* id -> dense idx */;

// CSR by counting sort on the source's dense index.
let mut offsets = vec![0u32; n + 1];
for &(s,_) in &pairs { offsets[index[&s] as usize + 1] += 1; }
for i in 0..n { offsets[i+1] += offsets[i]; }         // prefix sum

let mut adj = vec![0u32; pairs.len()];
let mut cursor = offsets.clone();
for &(s,d) in &pairs {
    let si = index[&s] as usize;
    adj[cursor[si] as usize] = index[&d];
    cursor[si] += 1;
}
```

Cost is `O(V + E)` plus the densify sort. The result is a compact `{ids, index, offsets, adj}` — the `GraphView`.

10.3 Consistency: it is a snapshot

`view()` pins an MVCC snapshot for the duration of the scan, so the CSR is built from **one consistent instant** of the graph. Writers keep committing edges; the `GraphView` is an immutable copy and does not change. That is what the test asserts:

```
let v = g.view()?;
let before = v.edge_count();
g.add_edges([(6,7,1.0),(7,8,1.0)])?;    // writes after the view was taken
assert_eq!(v.edge_count(), before);      // the snapshot view is unchanged
assert_eq!(g.view()?.edge_count(), before + 2); // a fresh view sees them
```

10.4 Memory: the honest ceiling

The CSR lives in RAM. `offsets` is `4·(V+1)` bytes and `neighbors` is `4·E` bytes — about **4 bytes per edge** plus the id maps. A 100-million-edge graph is ~0.4 GB of CSR; a billion edges is a few GB. Graph algorithms are memory-bound, and this is the same ceiling as ChakraDB's resident index: "fits in a big server's RAM" is in scope, trillion-edge graphs are not. For graphs larger than memory, restrict the view to a subgraph (a k-hop neighborhood, a component) before running the algorithm.

10.5 Live Graph Analytics

This is the chapter that says *why ChakraDB's graph layer is different*, not just convenient. The difference is one word: **live.**

10.6 The problem with the alternatives

To run a graph algorithm you need a consistent view of the graph. The usual ways to get one all give something up:

- **Load into NetworkX / igraph.** You get a *dead static copy*. The moment you loaded it, it began to go stale. Re-analyzing the latest graph means reloading the whole thing.
- **A lock-based graph database (e.g. Neo4j).** A long analytical traversal contends with the writers mutating the graph; you either block ingest or read an inconsistent, partially-updated graph.
- **A separate OLAP copy (ETL the graph into a warehouse).** Now the analysis is minutes-to-hours stale, and you run two systems.

10.7 What ChakraDB does instead

```
sequenceDiagram
    participant Writers
    participant Clock as MVCC clock
    participant Alg as Algorithm (GraphView)
    Writers->>Clock: add edge @ CSN 100
    Alg->>Clock: view() pins snapshot S = 100
    Note over Alg: builds CSR from S; runs PageRank...
    Writers->>Clock: add edge @ CSN 101  (never blocked)
    Writers->>Clock: add edge @ CSN 102
    Note over Alg: still computing over S = 100 — a consistent graph
    Alg-->>Alg: result reflects exactly the graph at S = 100
```

An algorithm builds its CSR from **one MVCC snapshot** and runs to completion over that consistent instant. Meanwhile writers keep committing edges — **never blocked**, because readers and writers share nothing but the append-only clock. The GC watermark holds the snapshot's versions alive for the algorithm's duration even as newer data is compacted.

The result: you can run PageRank (or components, or a fraud traversal) **every minute over the latest consistent graph, while ingest never pauses** — in one embedded process. That is the combination the alternatives cannot offer.

10.8 The pattern in code

```rust
// A background loop: recompute influence over the live graph, on a cadence.
loop {
    let view = graph.view()?;              // consistent snapshot; ingest continues
    let ranks = view.pagerank(20, 0.85);
    publish_top_influencers(&ranks);       // serve results
    sleep(Duration::from_secs(60));
}
```

Each `view()` is a fresh consistent snapshot. The writers feeding `graph` never notice the analytics running.

10.9 Why it composes with the rest of the database

Because the graph *is* tables, graph results compose with SQL and transactions over the **same** live data:

- Score an entity with a graph traversal **and** join it to its transactional row in one consistent snapshot.
- Ingest an event (transactional), update the graph edge (transactional), and let the next analytics pass pick it up — no cross-system consistency to reason about.

This is HTAP extended to graphs: **T + A + G over one snapshot clock.** The fraud case study works it end to end.

> **The honest scope.** v1 *recomputes* an algorithm over a snapshot on demand; it does not yet *maintain* live results incrementally as edges change. Incremental graph algorithms (streaming PageRank, incremental components) are a roadmap item — a v2 that leans even harder on this live-mutation story.

Chapter 11

Graph Algorithms

A journey of a thousand miles begins with a single step.

— Lao Tzu

All of ChakraDB's graph algorithms run over a `GraphView` — the immutable CSR snapshot. Because the view is a consistent copy taken from one MVCC snapshot, every algorithm below runs **concurrently with ingest**: writers keep adding edges while the algorithm computes over a stable graph. Each is a compact Rust kernel; this chapter gives the method, the pseudocode, and the complexity.

Throughout, `V` = number of nodes, `E` = number of directed edges, `d(u)` = out -degree of `u`.

11.1 Breadth-first search & shortest path

Unweighted shortest paths — "degrees of separation," reachability, k-hop neighborhoods — are BFS over the out-edges.

```
flowchart LR
    S((start)):::f --> A:::f1 --> B:::f2
    S --> C:::f1
    C --> B
    C --> D:::f2
    classDef f fill:#8ecae6; classDef f1 fill:#bde0fe; classDef f2 fill:#e7f0ff;
```

```
bfs(start):
  depth[*] = ∞;  depth[start] = 0;  queue = [start]
  while queue not empty:
    u = queue.pop_front()
    for v in neighbors(u):
      if depth[v] == ∞:
        depth[v] = depth[u] + 1
        queue.push_back(v)
  return depth
```

`shortest_path(a, b)` is the same wave with a `parent[]` array; on reaching `b`, it walks parents back to `a`. **Complexity `O(V + E)`**, one queue, one visited array.

```
assert_eq!(view.shortest_path(1, 4), Some(vec![1, 3, 4]));
```

11.2 PageRank

The influence/importance score: a node is important if important nodes point at it. ChakraDB computes it by **power iteration**, scattering each node's rank across its out-edges.

```
pagerank(iterations, d):            # d = damping, ~0.85
  rank[*] = 1/V
  repeat `iterations` times:
    dangling = Σ rank[u] for u with d(u) == 0
    base = (1-d)/V + d·dangling/V     # teleport + redistributed dead-ends
    next[*] = base
    for u in 0..V:
      if d(u) > 0:
        share = d · rank[u] / d(u)
        for v in neighbors(u): next[v] += share
    rank = next
  return rank                         # sums to ~1
```

The **dangling-node** handling matters: nodes with no out-edges would leak probability mass; their rank is collected and redistributed uniformly through `base`, so the scores stay a proper distribution. **Complexity `O(iterations · E)`.**

```
// A star where 2,3,4,5 all point at 1 → node 1 ranks highest.
let pr = view.pagerank(30, 0.85);
```

Personalized PageRank (recommendation, "similar to X") is the same iteration with the teleport vector concentrated on a seed set instead of uniform — on the roadmap.

11.3 Connected components

"Which nodes belong to the same island?" ChakraDB computes **weakly-connected components** (edges treated as undirected) with a **union-find** (disjoint-set) structure using path-halving.

```
connected_components():
  parent[u] = u for all u
  for each edge (u, v):
    union(u, v)          # merge the two sets
  relabel roots to 0..k
  return node -> component id
```

Union-find with path compression runs in near-linear **`O(E · α(V))`** time, where `α` is the inverse-Ackermann function (effectively constant). Strongly-connected components (Tarjan) are a roadmap addition.

```
let c = view.connected_components();
```

```
assert_eq!(c[&1], c[&4]);   // same island
assert_ne!(c[&1], c[&5]);   // different island
```

11.4 Triangle counting

Triangles measure local density — community strength, clustering coefficient, spam detection. ChakraDB counts them on the **undirected** graph by sorted-neighbor intersection with an ordering trick that counts each triangle exactly once.

```
triangle_count():
  build sorted, de-duplicated undirected adjacency
  total = 0
  for u in 0..V:
    for v in neighbors(u) with v > u:
      total += | { w in adj(u) ∩ adj(v) : w > v } |   # sorted two-pointer
  return total
```

Requiring `u < v < w` means the triangle `{u,v,w}` is counted once, not six times. **Complexity `O(E^{1.5})`** in the worst case (the classic node-iterator bound).

```
// Triangle 1-2-3, plus a dangling edge 3-4:
assert_eq!(view.triangle_count(), 1);
```

11.5 Degree & neighborhoods

The cheapest signals need no CSR at all:

- `out_degree(node)` — the length of the node's neighbor range.
- `out_neighbors(node)` — a single pruned range scan, live (no snapshot copy needed).
- k-hop neighborhood — BFS truncated at depth k.

11.6 Systemic risk: the Eisenberg–Noe clearing vector

Reading each directed edge `i → j` of weight `w` as a *liability* — `i` owes `j` the amount `w` — `eisenberg_noe(external_assets)` computes the clearing payment vector that models **default contagion**: who can pay, who defaults, and who a cascade drags under. It is the canonical systemic-risk model, solved by a monotone Picard iteration to the greatest clearing vector, and it powers the Counterparty Risk case study. Full derivation there.

11.7 Link prediction & recommendations

Two primitives turn the graph into a recommender (Recommendations case study):

- **`adamic_adar(a, b)`** — a link-prediction score summing `1/ln deg(z)` over shared neighbours `z`: rare, niche connections count for more than popular ones. On a user↔item graph it is "you might also like."
- **`recommend(seed, k)`** — personalized PageRank seeded at `seed`, with the seed's existing neighbours and zero-signal candidates removed, returning the top-`k` remainder. Collaborative filtering as a random-walk-with-restart, in one call.

11.8 The algorithm menu

Every algorithm below is built into the core (`src/graph.rs`) and available from Rust and Python, over a consistent `GraphView` snapshot.

Family	Algorithms
Traversal & paths	`bfs`, `shortest_path`, `dijkstra`, `weighted_shortest_path`, `topological_order`
Centrality	`pagerank`, `personalized_pagerank`, `degree_centrality`, `closeness_centrality`, `betweenness_centrality` (Brandes)
Community & structure	`connected_components` (WCC), `strongly_connected_components` / `laundering_cycles`, `label_propagation`, `k_core`, `triangle_count`
Similarity & link prediction	`common_neighbors`, `jaccard_similarity`, `adamic_adar`, `recommend`
Systemic risk	`eisenberg_noe` (clearing vector / default cascade)
Degrees	`in_degree`, `out_degree`, `in_neighbors`, `out_neighbors`

Deliberately **not** yet in core (expensive or specialized — candidates for a later release): Louvain/Leiden modularity, exact diameter, general subgraph isomorphism, Johnson's all-simple-cycles, temporal motifs, and max-flow/min-cut. See the design exploration for the roadmap.

Part V

Reactive

Chapter 12

Change Streams & Materialized Workers

You can never step into the same river twice, for new waters are always flowing on to you.

— *Heraclitus*

A traditional database is passive: you ask a question, it answers. The systems this book cares about — real-time AML, live risk — are *reactive*: the instant a transaction commits, something must happen. ChakraDB closes that loop inside the engine, on one consistent live dataset, without ever slowing the writer. This chapter is the machinery: a committed-change stream, workers that maintain derived state from it, and the transports that carry it across processes.

12.1 Why not in-SQL triggers

The obvious design — `CREATE TRIGGER` running a stored procedure inside the transaction — is the wrong one for this workload. A synchronous trigger runs user code (often Python, under the GIL) in the **writer's critical section**, so it serializes the very ingest that ChakraDB is built to keep fast. Worse, it couples the writer's latency and failure to arbitrary application logic.

ChakraDB instead exposes the model an event-driven system actually wants: a **post-commit change stream** (change-data-capture, CDC). Changes are delivered *after* they commit, on a separate thread — the writer never waits for a reader.

12.2 The change stream

A `CdcBackend` decorates the engine's backend and publishes a `Change` for every committed `INSERT` / `UPDATE` / `DELETE`, carrying the operation, the commit sequence number (CSN), and the old and new row images:

```
pub struct Change {
    pub table: String,
```

```
    pub op: ChangeOp,          // Insert | Update | Delete
    pub csn: Csn,              // total commit order
    pub columns: Arc<Vec<String>>,
    pub old: Option<Vec<Value>>,   // pre-image; None for Insert
    pub new: Option<Vec<Value>>,   // post-image; None for Delete
}
```

Because the wrap sits *above* the write path, it adds nothing to the engine's hot locks. An `INSERT` — the ingest hot path — pays only a channel send; the old-row pre-image for `UPDATE`/`DELETE` is read outside any core lock. This reuses the two things ChakraDB already has and most engines lack: an ordered commit log (the WAL, stamped with CSN) and cheap non-blocking snapshots.

Wire it up and subscribe:

```
let cdc = Cdc::new();
let engine = SqlEngine::with_backend(CdcBackend::wrap(db, cdc.clone()));
let stream = cdc.subscribe(Some("transactions"));   // one table, or None for all

while let Some(batch) = stream.recv() {              // one commit's changes, in order
    for change in &batch {
        if change.op == ChangeOp::Insert {
            react(change.new.as_ref().unwrap());
        }
    }
}
```

Delivery is **at-least-once** and **in commit order**; a rolled-back transaction never appears. The stream is a *pull* primitive — the consumer owns its thread, cadence, and backpressure.

ALGORITHM — publishing a committed change

```
On a mutating call m (insert/upsert/update/delete) through CdcBackend:
1  if m is update/delete: old ← inner.get_latest(key)      ▷ pre-image, no core
   ↪ lock
2  csn ← inner.apply(m)                                    ▷ the real write (+ WAL)
3  op  ← Insert | Update | Delete   (upsert: Update iff old present)
4  publish Change{ table, op, csn, columns, old, new }     ▷ after commit, off the
   ↪ write path
```

12.3 The Python hook

In Python the stream is a callback, and the connection is CDC-wrapped by default. `on_change` returns a `Subscription` whose lifecycle the caller owns:

```
def react(old, new):                    # dicts (column → value), or None
    if new and new["amount"] >= 9000:
        alert(new["src"], new["dst"])

sub = conn.on_change("transactions", react)
...                                      # worker runs on its own thread
sub.close()                             # stop it — or use it as a context manager
```

The callback fires after commit, so it never blocks the writer — the whole engine, including the graph algorithms, reachable from a hook in a few lines.

12.4 Materialized workers

Most reactions are not one-shot: they *maintain state* — a running aggregate, a graph projection, a set of alerts. That is a **materialized worker**: a named, incrementally-maintained derivation of the data. You implement *what* is maintained; the runtime owns the loop, commit ordering, cursor tracking, and lifecycle.

```
pub trait MaterializedWorker: Send + 'static {
    fn apply(&mut self, change: &Change);      // fold one change into state
    fn on_commit(&mut self, _csn: Csn) {}      // periodic hook (heavier passes)
}
```

```
let m = cdc.register("aml-detector", Some("transactions"), Worker::new());
m.query(|w| w.current_exposure.clone());       // read the derived state any time
m.cursor();                                  // how far it has consumed (a resume point)
m.stop();                                      // client owns the lifecycle
```

This is the disciplined worker primitive — **a function of the data**, not an application host. A worker consumes changes, maintains state, exposes it for query, and writes derived rows back; it does not open sockets or run arbitrary services. That boundary is what keeps ChakraDB a database and not an app server.

12.4.1 The registry

Named workers are tracked and observable — the control surface for a fleet:

```
for w in cdc.workers() {
    // w.name, w.table, w.cursor (CSN), w.running
}
cdc.stop_worker("aml-detector");
```

The CSN cursor is the key to durability: on restart, rebuild the worker's state from a snapshot and resume the stream from the last cursor — no missed or double-counted change, a hard requirement for compliance workloads.

12.4.2 The three-tier pattern

Firing per event rewards *incremental* work. The case studies use a tiered model so one embedded engine keeps up with millions of events per hour:

Tier	Runs	Scope	Example
T0	every committed change	O(1)–O(degree)	fan-in counter, limit check
T1	micro-batch (seconds)	touched subgraph	bounded cycle search
T2	periodic (minutes)	whole graph, on a `view()`	PageRank, Eisenberg–Noe, VaR

T0 keeps up with the firehose; T2 is heavy but rare and runs on an MVCC snapshot, so it never blocks ingest. That non-blocking split is the difference between real-time reaction and an overnight batch.

12.5 Transports: in-process, files, and Kafka

A `ChangeSink` carries the stream beyond the local thread:

```rust
pub trait ChangeSink: Send + Sync { fn emit(&self, batch: &[Change]); }
```

- **In-process channel** (default): lowest latency, single node — a worker in the same process, as the streaming case studies run.
- **`JsonlSink`**: appends each change as one JSON line to a file — a broker-less **cross-process** change log. A separate worker process tails the file and consumes the stream, with no shared memory and no database lock.
- **Kafka / Redpanda / NATS**: implement `emit` to produce each `change.to_json()` keyed by a partition key (e.g. account id). *N* workers then consume partitions in parallel for horizontal scale-out — the same worker code, a different transport.

```
flowchart LR
    W["committed writes"] --> CDC["change stream (CSN-ordered)"]
    CDC --> S{"ChangeSink"}
    S -->|in-process| A["worker (same process)"]
    S -->|JsonlSink| B["worker (separate process, tails the log)"]
    S -->|Kafka: key=hash(entity)| C["workers ×N (sharded)"]
```

The scaling story is one line: **resident workers for single-node hot state; Kafka-fanned workers for scale-out — the same derivation either way.**

12.6 Where it leads

Two complete systems are built on this machinery: the Real-Time AML and Counterparty Credit & Market Risk case studies. Each generates its own synthetic feed, reacts to every committed row through a registered materialized worker, and sustains 150+ million events per hour on a single node — detection running concurrently with ingest, because in ChakraDB the reader never blocks the writer.

Part VI

Getting Started

Chapter 13

Installation & Building

The beginning is the most important part of the work.
— *Plato,* The Republic

ChakraDB is an embedded database: there is no server to install, no daemon to run. You add it to a Rust project as a crate, or install the Python package and `import chakradb`. This chapter gets you from a clean checkout to a running build.

13.1 Prerequisites

- **Rust** 1.75 or newer (`rustup` recommended).
- A C toolchain (for a handful of transitive native dependencies).
- **Python** 3.9+ and `maturin` *only* if you want the Python bindings.

13.2 Building the core (Rust)

The core builds in two flavours, selected by Cargo features:

Build	Command	What you get
Interpreter only	`cargo build --no-default-features`	The full storage engine, SQL interpreter, MVCC, and the graph library — no analytical vectorized engine. Small and fast to compile.
With DataFusion	`cargo build --features datafusion`	Adds the vectorized analytical engine and the HTAP query router.

Run the test suite the same way:

```
cargo test --no-default-features          # core + graph
cargo test --features datafusion          # + analytical path
```

Why two profiles? The interpreter path is what powers the transactional workload and the graph engine; it has no heavy dependencies and compiles in seconds. DataFusion is bought in only when you want the analytical half of HTAP. Every algorithm in Part IV — Graph is available in *both* profiles.

13.3 Adding ChakraDB to a Rust project

```
# Cargo.toml
[dependencies]
chakradb = { path = "../chakradb", default-features = false }
```

Then, in code:

```
use chakradb::{Database, SqlEngine};
use std::sync::Arc;

let db = Arc::new(Database::new());     // in-memory
let sql = SqlEngine::new(db.clone());
sql.run("CREATE TABLE t (id INTEGER PRIMARY KEY, v INTEGER)")?;
```

For a durable, on-disk database, open a `Storage` over a directory instead — see Your First Database (Rust).

13.4 Building the Python bindings

The Python package is a thin PyO3 extension over the same core. From `bindings/python`:

```
cd bindings/python
maturin develop --release        # builds the extension into your venv
# or, for a wheel you can install elsewhere:
maturin build --release
```

Once built, the package is importable anywhere:

```
import chakradb
conn = chakradb.connect(":memory:")
```

The Python driver implements PEP 249 (DB-API 2.0) and exposes the graph engine via `conn.graph(name)` — see A Graph in Five Minutes.

13.5 Running the examples

Two end-to-end examples ship in the repository:

```
# Rust: a full real-time AML system over synthetic data
cargo run --release --example aml_realtime --no-default-features

# Python: the same application through the client bindings
python examples/aml_app.py
```

Both generate their own data, run the graph-analytics ensemble, and verify that every planted laundering typology is detected. They are the reference for the Real-Time AML case study.

13.6 Your First Database (Rust)

This chapter walks through a complete Rust program: create a durable database, define a table with real constraints and types, insert rows in a transaction, and query them back. Every feature here is exercised by the test suite.

13.7 An in-memory database

The smallest possible program:

```rust
use chakradb::{Database, SqlEngine};
use std::sync::Arc;

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let db = Arc::new(Database::new());
    let sql = SqlEngine::new(db.clone());

    sql.run("CREATE TABLE accounts (
        id      INTEGER PRIMARY KEY,
        owner   VARCHAR(64) NOT NULL,
        balance DECIMAL(14,2) NOT NULL DEFAULT 0.00
    )")?;

    sql.run("INSERT INTO accounts VALUES (1, 'ada', 500.00)")?;
    sql.run("INSERT INTO accounts VALUES (2, 'grace', 1250.50)")?;

    let rows = sql.query("SELECT id, owner, balance FROM accounts ORDER BY id")?;
    for r in rows {
        println!("{r:?}");
    }
    Ok(())
}
```

`SqlEngine::run` executes a statement (DDL or DML) and returns an `Outcome`; `SqlEngine::query` is a convenience that returns rows already rendered as `Vec<Vec<String>>`.

13.8 Types and constraints are enforced

ChakraDB is not a stringly-typed store. The declared types and constraints are checked at write time:

```rust
// Rejected: NOT NULL violated.
assert!(sql.run("INSERT INTO accounts (id, owner) VALUES (3, NULL)").is_err());

// Rejected: DECIMAL(14,2) — an over-precise literal is a constraint error, not
```

```
// a silent rounding.
assert!(sql.run("INSERT INTO accounts VALUES (3, 'lin', 1.234)").is_err());

// Rejected: VARCHAR(64) length is enforced.
// Rejected: duplicate PRIMARY KEY.
assert!(sql.run("INSERT INTO accounts VALUES (1, 'dup', 0.00)").is_err());
```

Money is stored as **exact** fixed-point `DECIMAL`, never binary floating point — see Exact Decimal Arithmetic.

13.9 Transactions

By default each statement autocommits. Wrap several in a transaction for all-or-nothing semantics under snapshot isolation:

```
sql.begin()?;
sql.run("INSERT INTO accounts VALUES (10, 'noether', 300.00)")?;
sql.run("INSERT INTO accounts VALUES (11, 'hopper', 300.00)")?;
sql.commit()?;                       // both visible, atomically
// ... or sql.rollback()? to discard the whole unit.
```

Readers never block writers and writers never block readers: a transaction sees a consistent MVCC snapshot taken at its start.

13.10 A durable, on-disk database

Swap the in-memory `Database` for a `Storage` opened over a directory. Nothing else in your code changes — the SQL surface is identical:

```
use chakradb::{PosixIo, SqlEngine};
use chakradb::storage::{Storage, StorageConfig};
use std::sync::Arc;

let io = PosixIo::new("/var/lib/myapp")?;
let storage = Storage::open(Arc::new(io), StorageConfig::default())?;
let sql = SqlEngine::durable(Arc::new(storage));
```

Writes are made durable through a write-ahead log with group commit, and the engine recovers automatically on the next open — see Durability and Crash Recovery.

13.11 Where to go next

- Python in Five Minutes — the same engine from Python.
- A Graph in Five Minutes — treat your tables as a graph.
- The SQL Reference — the full statement surface.

13.12 Python in Five Minutes

The Python bindings wrap the exact same engine as the Rust core. The SQL surface is exposed through a standard PEP 249 (DB-API 2.0) driver, so if you have used `sqlite3` you already know the shape of it.

13.13 Connect, execute, fetch

```
import chakradb

conn = chakradb.connect(":memory:")          # or a directory path for durability
cur = conn.execute("""
    CREATE TABLE accounts (
        id      INTEGER PRIMARY KEY,
        owner   VARCHAR(64) NOT NULL,
        balance DECIMAL(14,2) NOT NULL DEFAULT 0.00
    )""")

conn.execute("INSERT INTO accounts VALUES (1, 'ada', 500.00)")
conn.execute("INSERT INTO accounts VALUES (2, 'grace', 1250.50)")
conn.commit()

for row in conn.execute("SELECT id, owner, balance FROM accounts ORDER BY id"):
    print(row)          # (1, 'ada', '500.00')  ...
```

`connect`, `Connection.execute`, `Cursor.fetchone/fetchmany/fetchall`, `commit`, `rollback`, and the full exception hierarchy (`IntegrityError`, `ProgrammingError`, `OperationalError`, …) all behave exactly as PEP 249 specifies.

13.14 Parameters and transactions

Parameters use the `qmark` style; transactions are explicit around a unit of work:

```
conn.execute("INSERT INTO accounts VALUES (?, ?, ?)", (3, 'lin', 42.00))

try:
    conn.execute("BEGIN")  # or rely on autocommit-off via .execute + .commit
    conn.execute("INSERT INTO accounts VALUES (10, 'noether', 300.00)")
    conn.execute("INSERT INTO accounts VALUES (11, 'hopper', 300.00)")
    conn.commit()
except chakradb.IntegrityError as e:
    conn.rollback()
    print("rejected:", e)
```

Type and constraint violations surface as `IntegrityError`; malformed SQL as `ProgrammingError`; a write–write conflict as `OperationalError`.

13.15 The graph engine, from Python

Any table can be viewed as a graph. `conn.graph(name)` returns a handle backed by table `name` in the same database; edges are ordinary transactional rows:

```
g = conn.graph("payments")
g.add_edges([(1, 2, 100.0), (2, 3, 250.0), (3, 1, 90.0)])   # (src, dst, weight)

view = g.view()                          # one consistent snapshot for analytics
print(view.laundering_cycles())          # [[1, 2, 3]] — a round-trip ring
print(view.pagerank())                   # {1: 0.33, 2: 0.33, 3: 0.33}
print(view.personalized_pagerank(seeds=[1]))
```

Every algorithm returns plain Python `dict`/`list` values keyed by node id. The full catalogue is in A Graph in Five Minutes and Graph Algorithms.

13.16 A complete application

`examples/aml_app.py` is a full real-time anti-money-laundering system in ~300 lines of Python: it builds a synthetic payment network, runs an ensemble of graph detectors, and ranks suspicious accounts. Walk through it in the Real-Time AML case study.

13.17 A Graph in Five Minutes

ChakraDB has a graph engine built into the core — not a bolt-on, not a separate store. A graph *is* a table of edges, clustered so that a node's neighbours sit together on disk; analytics run over a consistent MVCC snapshot of that table. This chapter is the whirlwind tour.

13.18 Open a graph and add edges

A graph is backed by a named table in an ordinary database. In Rust:

```
use chakradb::{Database, Graph};
use std::sync::Arc;

let db = Arc::new(Database::new());
let g = Graph::open(db.clone(), "transfers")?;

// (src, dst, weight) — inserts are upserts, and fully transactional.
g.add_edges([(1, 2, 100.0), (2, 3, 250.0), (3, 1, 90.0)])?;
```

In Python, through an existing connection:

```
g = conn.graph("transfers")
g.add_edges([(1, 2, 100.0), (2, 3, 250.0), (3, 1, 90.0)])
```

Because edges are just rows, the *same* data is visible to SQL (`SELECT ... FROM transfers`) and to the graph engine simultaneously — that is the HTAP promise applied to graphs.

13.19 Freeze a snapshot, then compute

All algorithms run against a `view()` — an immutable in-memory CSR snapshot. Take it once and a whole pipeline sees one coherent graph, even while writers keep appending edges:

```
let view = g.view()?;
println!("{} nodes, {} edges", view.node_count(), view.edge_count());

let dist   = view.bfs(1);                      // hops from node 1
let path   = view.shortest_path(1, 3);         // Some([1, 2, 3])
let cost   = view.weighted_shortest_path(1, 3); // Dijkstra with weights
let rank   = view.pagerank(20, 0.85);          // {node: score}
let rings  = view.laundering_cycles();         // non-trivial SCCs
```

The Python surface is identical, returning dicts and lists:

```
view = g.view()
view.bfs(1)                       # {1: 0, 2: 1, 3: 1}
view.shortest_path(1, 3)          # [1, 2, 3]
view.pagerank()                   # {1: 0.33, 2: 0.33, 3: 0.33}
view.laundering_cycles()          # [[1, 2, 3]]
view.personalized_pagerank([1])   # risk spread from a seed set
```

13.20 The algorithm catalogue

Every one of these is built into the core and available from both Rust and Python. See Graph Algorithms for the derivations.

Category	Algorithms
Traversal & paths	`bfs`, `shortest_path`, `dijkstra`, `weighted_shortest_path`, `topological_order`
Centrality	`pagerank`, `personalized_pagerank`, `degree_centrality`, `closeness_centrality`, `betweenness_centrality`
Community & structure	`connected_components`, `strongly_connected_components`, `laundering_cycles`, `label_propagation`, `k_core`, `triangle_count`
Systemic risk	`eisenberg_noe` (clearing vector / default cascade)
Similarity & link prediction	`common_neighbors`, `jaccard_similarity`, `adamic_adar`, `recommend`
Degrees	`in_degree`, `out_degree`, `in_neighbors`, `out_neighbors`

13.21 Snapshots are stable under writes

The defining property — an analysis is reproducible because its view does not move under it:

```rust
let v = g.view()?;
let before = v.edge_count();
g.add_edges([(9, 10, 1.0)])?;           // keep writing after the view is taken
assert_eq!(v.edge_count(), before);     // the snapshot did not change
assert_eq!(g.view()?.edge_count(), before + 1);  // a fresh view sees the write
```

13.22 A real application

The Real-Time AML case study composes these primitives into a working fraud-detection system. The runnable code is `examples/aml_realtime.rs` (Rust) and `examples/aml_app.py` (Python).

Part VII

Case Studies

Chapter 14

Case Study: A Real-Time Anti-Money-Laundering System

Follow the money.

— Deep Throat, All the President's Men

This is the chapter where every part of ChakraDB is used at once, on a workload that genuinely needs all of it: **anti-money-laundering (AML)**. Money laundering is a *graph* problem wearing a *transactional* disguise — a stream of individually innocuous transfers whose *shape*, taken together, is the crime. Detecting it in real time means ingesting transactions transactionally, scoring them analytically, and traversing the entity graph they form — over one consistent view, as the data arrives. That is exactly the HTAP-plus-graph workload ChakraDB is built for.

We build a working detection engine: the schema, the ingestion path, a detector for each of the major laundering typologies (mapped onto a ChakraDB primitive), the scoring pipeline that combines them, and the case-management output — all in one embedded process, over live data.

14.1 The problem

Laundering has three classic stages, and each leaves a different footprint in the data:

- **Placement** — dirty cash enters the system (many small deposits). Footprint: *structuring / smurfing* — many small credits, kept under reporting thresholds, fanning **into** one account.
- **Layering** — the money is moved through a maze of transfers to obscure its origin. Footprint: *round-tripping and chains* — directed **cycles** and long paths through many accounts, often rapidly.
- **Integration** — the laundered money returns to the criminal as apparently legitimate funds. Footprint: *mule networks* — dense clusters of accounts that concentrate flow toward a few beneficiaries.

```
flowchart LR
    subgraph P["Placement (structuring)"]
```

```
    D1(( )) --> M[Mule A]
    D2(( )) --> M
    D3(( )) --> M
    D4(( )) --> M
  end
  subgraph L["Layering (round-trip)"]
    M --> B[Acct B] --> C[Acct C] --> M
  end
  subgraph I["Integration"]
    C --> Z[Beneficiary]
  end
  classDef s fill:#f4c430; class M,Z s;
```

No single transfer is suspicious. The *fan-in* at the mule, the *cycle* B→C→A→B, and the *concentration* toward the beneficiary are — and each is a graph query.

14.2 Why one engine

The traditional AML stack fragments this: an OLTP core banking system for the transactions, a nightly batch to a warehouse for the rules, and a separate graph database for network analysis — reconciled by ETL. The consequences are exactly the ones that matter for AML: the graph is **hours stale**, so layering is caught after the money is gone; the systems disagree on *which* transactions are in scope; and running the network analysis contends with ingestion.

ChakraDB collapses the stack. Transactions, rule analytics, and graph traversal all read the **same MVCC snapshot** of the **same live data**, in one process, with writers never blocked by the scorer. The detector below runs *as transactions arrive*.

14.3 The schema

Two logical layers: the transactional records, and the entity/flow graph derived from them.

```
-- Customers and their risk attributes.
CREATE TABLE customers (
  id          INT PRIMARY KEY,
  name        VARCHAR(128) NOT NULL,
  country     VARCHAR(2),
  risk_rating VARCHAR(8) DEFAULT 'low',     -- low | medium | high
  is_pep      BOOLEAN DEFAULT false,        -- politically-exposed person
  onboarded   DATE
);

-- Accounts, each owned by a customer.
CREATE TABLE accounts (
  id          INT PRIMARY KEY,
  customer_id INT NOT NULL,
  opened      DATE,
  status      VARCHAR(8) DEFAULT 'active'
```

```
);

-- The transaction ledger — exact money, never rounded.
CREATE TABLE txns (
  id       INT PRIMARY KEY,
  src_acct INT NOT NULL,
  dst_acct INT NOT NULL,
  amount   DECIMAL(14,2) NOT NULL CHECK (amount > 0),
  ts       TIMESTAMP NOT NULL,
  channel  VARCHAR(12),                    -- wire | ach | card | crypto
  CHECK (src_acct <> dst_acct)
);

-- A watchlist of accounts already known to be bad (SARs filed, sanctions, ...).
CREATE TABLE known_bad (
  acct     INT PRIMARY KEY,
  reason   VARCHAR(32),
  added    DATE
);
```

The **flow graph** is derived: a node is an account, and a directed edge `src → dst` means "money moved from src to dst," weighted by amount. ChakraDB's `Graph` handle maintains it as an edges table keyed `(src, dst)` so adjacency is clustered (The Graph Model).

14.4 Ingestion

Each incoming transaction is one ACID transaction: the ledger row and the flow edge commit together, so the graph is never half-updated relative to the ledger.

```
use chakradb::{SqlEngine, Graph};

fn ingest(sql: &SqlEngine, flow: &Graph, t: &Txn) -> anyhow::Result<()> {
    sql.run(&format!(
        "BEGIN;
         INSERT INTO txns VALUES ({}, {}, {}, {:.2}, '{}', '{}');
         COMMIT;",
        t.id, t.src_acct, t.dst_acct, t.amount, t.ts, t.channel
    ))?;
    // The flow edge, weighted by amount (idempotent upsert on (src,dst)).
    flow.add_edge(t.src_acct, t.dst_acct, t.amount)?;
    Ok(())
}
```

`DECIMAL(14,2)` keeps amounts exact; the `CHECK`s reject nonsense before it reaches the log. Ingestion never pauses for the detector — the wedge.

14.5 The detection engine

Each laundering typology maps onto a ChakraDB primitive. We take **one snapshot** — a SQL view of the current state and a `GraphView` (CSR) built from the same instant (Live Graph Analytics) — so every detector sees a consistent graph.

```
let view = flow.view()?;   // consistent CSR snapshot; ingestion continues
```

14.5.1 1. Structuring / smurfing — graph fan-in + SQL velocity

Many small credits fanning **into** one account, kept under a reporting threshold, in a short window. Fan-in is `in_degree`; the threshold and velocity are SQL.

```
// Candidate mules: high fan-in of small, sub-threshold credits in 24h.
fn structuring_suspects(sql: &SqlEngine, view: &GraphView, acct: NodeId) -> bool {
    let fan_in = view.in_degree(acct);                 // graph: many senders
    let small_credits: i64 = sql.query(&format!(
        "SELECT COUNT(*) FROM txns
         WHERE dst_acct = {acct} AND amount < 10000.00
           AND ts > NOW_MINUS_24H"))?[0][0].parse().unwrap_or(0);
    fan_in >= 8 && small_credits >= 8                  // tune per institution
}
```

14.5.2 2. Round-tripping / layering — laundering cycles (SCC)

Funds that can return to their origin through a chain of transfers form a **directed cycle** — a strongly-connected component of size ≥ 2. `laundering_cycles` finds them directly (Graph Algorithms).

```
// Every set of accounts among which money can circulate.
let cycles: Vec<Vec<NodeId>> = view.laundering_cycles();
for ring in &cycles {
    alert(Alert::layering(ring, "funds can round-trip among these accounts"));
}
```

A single `laundering_cycles()` call replaces the recursive path-search that a relational-only system cannot express (SQL has no recursion in ChakraDB's surface — which is *why* the algorithm lives in the engine).

14.5.3 3. Mule networks — connected components + centrality

A laundering operation is a **dense cluster** that concentrates flow toward a few beneficiaries. Weakly-connected components isolate the clusters; PageRank finds the beneficiaries within one.

```
let comps = view.connected_components();               // isolate clusters
let ranks = view.pagerank(30, 0.85);                   // flow concentration
// A component with an unusually high-PageRank sink is a candidate mule network.
```

14.5.4 4. Risk propagation from known-bad — personalized PageRank

The most powerful signal: **proximity to accounts already known to be bad**. Seed personalized PageRank with the watchlist, and every account gets a risk score that is its exposure to the seeds — decaying with graph distance.

```
let seeds: Vec<NodeId> = sql.query("SELECT acct FROM known_bad")?
    .iter().map(|r| r[0].parse().unwrap()).collect();
let risk = view.personalized_pagerank(&seeds, 40, 0.85);
// risk[&acct] is high for accounts transacting near the watchlist — even
// several hops away, which flat rules miss entirely.
```

14.5.5 5. Rapid movement & fan-out — SQL + graph out-degree

Layering also shows as money that arrives and leaves almost immediately, and as *fan-out* (one account distributing to many). Velocity is a temporal SQL query; fan-out is `out_degree`.

```
let fan_out = view.out_degree(acct);                    // distribution
let dwell_ms: i64 = sql.query(&format!(
    "SELECT MIN(out.ts - inn.ts) FROM txns inn, txns out
     WHERE inn.dst_acct = {acct} AND out.src_acct = {acct}"))?[0][0].parse()?;
let rapid = dwell_ms < 60_000;                           // in-and-out under a minute
```

14.6 The scoring pipeline

The detectors combine into a single risk score per account, computed over one snapshot, and above a threshold they open a case:

```
fn score_accounts(sql: &SqlEngine, flow: &Graph) -> anyhow::Result<Vec<Case>> {
    let view = flow.view()?;                             // one consistent instant
    let seeds = known_bad_accounts(sql)?;
    let risk  = view.personalized_pagerank(&seeds, 40, 0.85);
    let cycles = view.laundering_cycles();
    let in_cycle: HashSet<NodeId> = cycles.iter().flatten().copied().collect();

    let mut cases = Vec::new();
    for &acct in view_accounts(&view) {
        let mut score = 0.0;
        let mut reasons = Vec::new();

        // (1) proximity to known-bad (the dominant term)
        if let Some(&r) = risk.get(&acct) {
            score += 100.0 * r;
            if r > SEED_PROXIMITY { reasons.push("near a watchlisted account"); }
        }
        // (2) in a laundering cycle
        if in_cycle.contains(&acct) {
            score += 40.0; reasons.push("member of a round-trip cycle");
        }
```

```
        // (3) structuring fan-in
        if structuring_suspects(sql, &view, acct) {
            score += 30.0; reasons.push("structuring: high small-credit fan-in");
        }
        // (4) fan-out distribution
        if view.out_degree(acct) >= 20 {
            score += 15.0; reasons.push("fan-out distribution");
        }

        if score >= CASE_THRESHOLD {
            cases.push(Case { acct, score, reasons, snapshot: view.node_count() });
        }
    }
    cases.sort_by(|a, b| b.score.partial_cmp(&a.score).unwrap());
    Ok(cases)
}
```

Because all four signals read the **same** `view` and the **same** SQL snapshot, the score is internally consistent — no detector is looking at a different instant of the graph, which in a fragmented stack is a constant source of false alerts.

14.7 Case management and SAR output

A case above threshold is written back as a queryable table — so investigators triage with plain SQL, joined to the customer records, over the same live data:

```
-- The investigator's queue: highest-risk cases with the human context.
SELECT c.name, c.country, c.is_pep, a.score, a.reasons
FROM   alerts a
JOIN   accounts   acct ON acct.id = a.acct
JOIN   customers  c    ON c.id = acct.customer_id
WHERE  a.score >= 80
ORDER  BY a.score DESC;
```

Confirmed cases feed back into `known_bad`, which **re-seeds** the personalized PageRank on the next pass — the system learns: each confirmed mule sharpens the risk propagation for the accounts around it.

14.8 Operating it live

```
sequenceDiagram
    participant TX as Transaction stream
    participant DB as ChakraDB
    participant SC as Scorer (every N seconds)
    participant INV as Investigator
    loop continuously
      TX->>DB: ingest (txn + flow edge, one commit)
    end
```

```
loop on a cadence
  SC->>DB: view() — consistent snapshot; ingestion never pauses
  SC->>SC: PPR + cycles + fan-in/out + velocity
  SC->>DB: write alerts table
end
INV->>DB: SQL over alerts JOIN customers (same live data)
```

- The **inline** checks (fan-in, velocity, watchlist membership) run per transaction using live adjacency (`in_degree`, `out_neighbors`) — cheap, no snapshot copy.
- The **network** analysis (cycles, PageRank, personalized PageRank) runs on a cadence over a `view()`; each pass is a fresh consistent snapshot while ingestion continues at full rate.
- Watch `Storage::stats()` for checkpoint lag and backpressure under peak load (Observability); back up the store on a schedule (Backup & Restore).

14.9 Scaling to millions per hour — the event-driven pipeline

The batch scorer above is correct but polls. A production AML system is *event-driven*: the instant a transaction commits, detection should react — and it must do so without ever slowing the ingest that feeds it. This is where ChakraDB's design pays off, and it is worth building out in full because it is the workload that sets ChakraDB apart.

14.9.1 The trigger: a committed-change stream

ChakraDB deliberately has **no in-SQL triggers**. Running user code (especially Python under the GIL) inside the writer's critical section would throttle the one thing that has to stay fast. Instead it exposes the model an event-driven system actually wants: a **stream of committed row changes** (change-data-capture).

A `CdcBackend` decorates the engine's backend and publishes a `Change {op, csn, old, new}` after each INSERT/UPDATE/DELETE commits — after the write is applied and, on the durable backend, after it is WAL-logged. An INSERT (the ingest hot path) pays only a channel send; the old-row image for UPDATE/DELETE is read outside any engine lock. Nothing touches the writer's hot locks.

In Rust, a subscription is a pull-based stream:

```rust
let cdc = Cdc::new();
let engine = SqlEngine::with_backend(CdcBackend::wrap(db, cdc.clone()));
let stream = cdc.subscribe(Some("transactions"));   // one table, or None for all

// on the worker thread:
while let Some(batch) = stream.recv() {              // one commit's changes
    for change in &batch {
        if change.op == ChangeOp::Insert {
            react(change.new.as_ref().unwrap());     // fire detectors
        }
    }
}
```

In Python it is a callback — the entire engine, including the graph algorithms, in a few lines:

```
def react(old, new):
    if new:
        worker.on_transaction(new)          # dict: {"src":…, "dst":…, "amount":…}

conn.on_change("transactions", react)       # fires after every committed write
```

Delivery is at-least-once and in commit order; a rolled-back transaction never appears. A `ChangeSink` trait leaves room for a Kafka transport (below) without changing a line of detector code.

14.9.2 The three-tier detection model

The trap every "fraud on a graph" system falls into is recomputing global graph algorithms per event. At millions of transactions per hour you cannot run PageRank on every insert. The pipeline splits detection into three tiers by *how much of the graph each needs* and *how fresh it must be*:

Tier	Runs	Scope	Cost	Catches
T0 — per-event	on every committed txn, at ingest rate	O(1)–O(degree)	counter bump, set lookup	fan-in threshold crossed, transacts-with-known-bad, fan-out
T1 — micro-batch	every few seconds, on the touched subgraph	k-hop neighbourhood of new edges	bounded	short laundering cycles (bounded length + Δt), local dense mule clusters
T2 — periodic	every few minutes, over a `view()` snapshot	whole graph	seconds, **off the write path**	global `personalized_pagerank`, full `strongly_connected_components` communities

T0 keeps up with the firehose because it never touches more than one node's neighbourhood. T2 is expensive but rare, and runs on an **MVCC snapshot** — so it never blocks ingest. That single fact is the difference between ChakraDB and a bolt-on stack.

The worker's T0 and T2 in pseudocode:

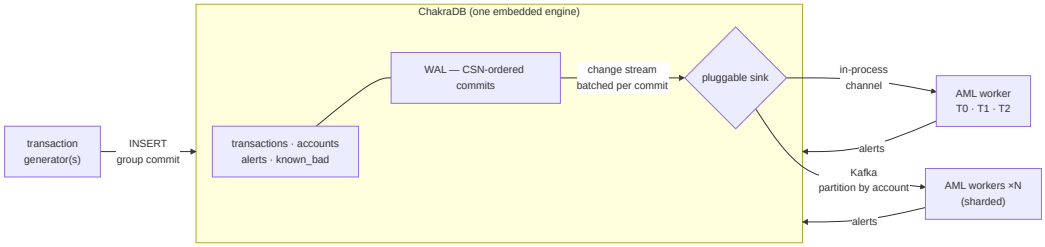
ALGORITHM — T0 (per committed transaction)

```
Input: change (src, dst, amount)
1  graph.add_edge(src, dst, amount)                ▷ mirror into payment graph
2  in_sources[dst].add(src);  fan_in ← |in_sources[dst]|
3  if 9000 ≤ amount < 10000: near_threshold[dst] += 1
4  out_targets[src].add(dst); fan_out ← |out_targets[src]|
5  if fan_in ≥ θ_in  and  near_threshold[dst] ≥ θ_near:  alert(dst, STRUCTURING)
6  if fan_out ≥ θ_out:                                    alert(src, FAN_OUT)
7  if src ∈ known_bad:                                   alert(dst, KNOWN_BAD)
```

ALGORITHM — T2 (every N events, over one snapshot)

```
1  view ← graph.view()                                  ▷ consistent MVCC snapshot
2  for ring in view.laundering_cycles():                ▷ non-trivial SCCs
3      for m in ring: alert(m, LAYERING_CYCLE)
4  risk ← view.personalized_pagerank(known_bad)          ▷ risk propagation
5  for acct in top_k(risk): alert(acct, HIGH_RISK_EXPOSURE)
```

14.9.3 The architecture



- **Ingest** rides ChakraDB's group commit; the writer is never the bottleneck.
- **The change stream** emits one batch per commit, CSN-stamped — ordered, replayable, resumable by cursor.
- **Transport is pluggable**: the in-process channel (default, µs latency, single node) or Kafka (partition key = `hash(account)`) so N workers consume in parallel and survive restarts — near-linear horizontal scale-out.
- **T2 runs on a `view()`** — a consistent snapshot — so heavy analytics never block the writer. A Neo4j + Postgres + Flink stack must ETL OLTP → graph → OLAP and can never give you one transactionally-consistent view; ChakraDB does it in one process, no copy.

14.9.4 Capacity: it is not close

The shipped in-process pipeline (`examples/aml_pipeline.rs`) runs a generator thread and the reacting worker concurrently over one engine. On a single machine it sustains, end to end — ingest *plus* per-event detection *plus* periodic global graph passes:

```
Ingest: 60,037 transactions in 1.39s
        = 43,196 txn/s  ≈  155 million txn/hour
Worker: reacted to 60,037 committed transactions via the change stream
All typologies detected live — pipeline verified.
```

The standalone generator (`examples/aml_gen.rs`) writes at ~190 million txn/hour. Against a target of "millions per hour," the engine has roughly two orders of magnitude of headroom — because readers never block writers, so the detection load and the ingest load do not contend. This is the axis the project competes on, made concrete.

14.9.5 Horizontal scale-out

Beyond a single node, the Kafka sink partitions the change stream by `hash(account)`. Each worker owns a shard of the payment graph and its incremental T0 state; the known-bad seed set is replicated to every shard, and cross-shard rings are escalated to a coordinator. The CSN cursor per partition makes each worker resume exactly where it left off after a restart — no missed or double-counted transaction, which is a hard compliance requirement.

14.9.6 The two implementations

The complete pipeline ships in both languages:

- **`examples/aml_pipeline.rs`** — the scalable reference: a generator thread and a tiered worker over one `SqlEngine`, reacting through the Rust `ChangeStream`.
- **`examples/aml_stream.py`** — the ergonomic mirror: the same tiers driven by `conn.on_change`. (While a T2 pass holds the GIL, ingest pauses briefly — which is precisely why the scalable worker is the Rust one.)
- **`examples/aml_gen.rs`** — the standalone producer CLI (`--db`, `--count`, `--seed`), the feed for the Kafka-partitioned topology.

14.10 What ChakraDB made possible

Laundering typology	ChakraDB primitive
Structuring / smurfing	`in_degree` (fan-in) + SQL velocity
Round-tripping / layering	`laundering_cycles` (SCC)
Mule networks	`connected_components` + `pagerank`
Proximity to known-bad	`personalized_pagerank(seeds)`
Rapid movement	temporal SQL
Fan-out distribution	`out_degree`

Every one of these ran over the **same live snapshot**, in **one embedded process**, while transactions kept arriving — the combination the fragmented AML stack cannot offer. The graph algorithms the system needed (reverse adjacency for fan-in, SCC for cycles, personalized PageRank for risk propagation) are built into ChakraDB's core (`src/graph.rs`), so the application is a few hundred lines of scoring logic, not a three-system integration project. **Follow the money — in real time, over one consistent view of it.**

14.11 The reference implementation

This case study ships as two runnable programs — the same system in each language, generating their own synthetic data (legitimate traffic with laundering rings deliberately injected) and *asserting* that every planted typology is caught:

```
cargo run --release --example aml_realtime --no-default-features   # Rust
python examples/aml_app.py                                          # Python
```

Both build a ~230-account payment network, run the five detectors over one `view()`, and emit a ranked SAR list. The output speaks for itself — signal cleanly separated from a sea of legitimate activity, with zero false positives:

```
── Detector 1 — Structuring (fan-in of near-threshold deposits) ──
  account  500: 12 distinct sources, 12 just-under-threshold deposits, $114,669 received

── Detector 2 — Layering (round-trip cycles / SCCs) ──
  ring #1: [700, 701, 702, 703]  — funds return to origin

── Detector 3 — Mule fan-out (distribution hubs) ──
  account  800: pays out to 20 distinct counterparties

── Detector 4 — Risk propagation (personalized PageRank from known-bad) ──
  seed(s) [800] → risk reached 66 downstream accounts.

── SAR candidates — fused risk ranking ──
   acct   score  reasons
    800    6.00  known-bad, mule-fan-out
    703    3.00  laundering-cycle
    700    3.00  laundering-cycle
    702    3.00  laundering-cycle
```

```
701    3.00  laundering-cycle
500    2.50  structuring-collector
```

The Rust program is the canonical reference; the Python program is the same logic through `conn.graph(...)`, showing that the *entire* engine — SQL, exact money, and the graph algorithms — is available to a client in a few hundred lines. **Follow the money — in real time, over one consistent view of it.**

Chapter 15

Case Study: Real-Time Counterparty Credit & Market Risk

The revolutionary idea that defines the boundary between modern times and the past is the mastery of risk.

— *Peter L. Bernstein,* Against the Gods

Banks still compute much of their risk **overnight**, in batch: the book is frozen, market data is snapshotted, and a grid churns until morning. By the time the numbers land, the market has moved. The prize everyone chases is **intraday, real-time risk** — exposure, limits, VaR, and systemic contagion recomputed as prices tick and trades book. That is precisely the shape of workload ChakraDB is built for, and this chapter builds it end to end.

It is the risk analogue of the AML case study: a live portfolio and a streaming market-data feed drive a **materialized worker** that reacts to every price tick through the change stream, over one consistent dataset, never blocking ingest. The runnable code is `examples/ccr_pipeline.rs` (Rust) and `examples/ccr_stream.py` (Python).

15.1 Two risk domains, one engine

Domain	Question	Cadence
Market risk	How much could the book lose from price moves? (MtM, VaR)	per tick + periodic
Counterparty credit risk (CCR)	If a counterparty defaults, what do we lose — and who else falls?	per trade + periodic

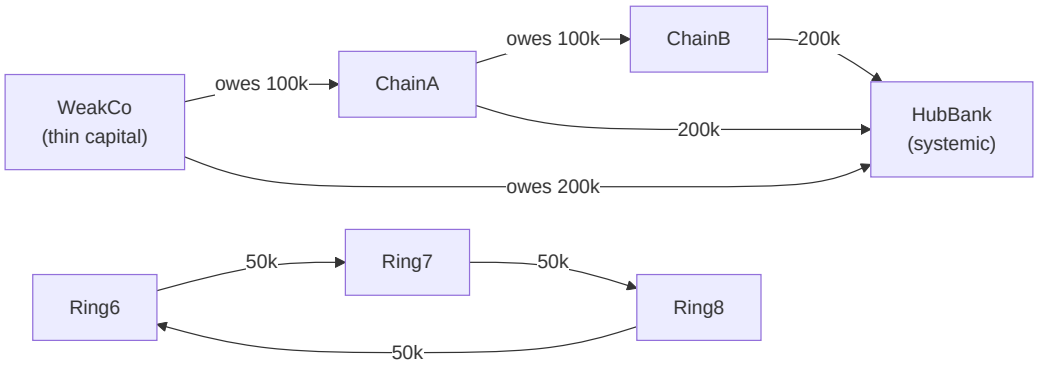
Both are functions of the *same* live data — the trades, the prices, and the web of who owes whom. Stitching them across an OLTP store, a market-data bus, a graph database, and an overnight risk

grid is the status quo; doing them together, in one embedded engine, on one consistent snapshot, is the ChakraDB thesis.

15.2 The exposure network is a graph

CCR is intrinsically a **network** problem, and this is where the built-in graph engine becomes the risk engine. Model the interbank system as a directed, weighted graph: a node is a legal entity; an edge `i → j` of weight `w` means *`i` owes `j` the amount `w`*. On that one structure:

Risk question	Built-in primitive
Who is systemically important (too-connected-to-fail)?	`pagerank` on the liability network
If X defaults, who is dragged under, and by how much?	**`eisenberg_noe`** (clearing vector / default cascade)
Hidden circular exposures = netting opportunities	`laundering_cycles` (SCCs)
Netting sets / collateral pools	`connected_components`
Concentration / Cover-2	top-k weighted degree



15.3 The schema

Trades and market data are ordinary tables; the exposure network is a graph over the same database.

```
CREATE TABLE counterparties (id INTEGER PRIMARY KEY, name VARCHAR(32),
    rating VARCHAR(4), external_assets DECIMAL(18,2));
CREATE TABLE trades (trade_id INTEGER PRIMARY KEY, counterparty INTEGER,
    instrument INTEGER, notional DECIMAL(18,2), trade_price DECIMAL(12,4));
```

```
CREATE TABLE market_data (tick_id INTEGER PRIMARY KEY, instrument INTEGER,
    price DECIMAL(12,4), ts TIMESTAMP);              -- the streaming feed
CREATE TABLE limits (counterparty INTEGER PRIMARY KEY, limit_amount DECIMAL(18,2));
CREATE TABLE interbank (id INTEGER PRIMARY KEY, debtor INTEGER, creditor INTEGER,
    amount DECIMAL(18,2));                            -- edges of the exposure graph
CREATE TABLE risk_alerts (id INTEGER PRIMARY KEY, entity INTEGER, kind VARCHAR(40));
```

Money is exact `DECIMAL`, never binary float — a hard requirement when the number is a P&L or an exposure (Exact Decimal Arithmetic).

15.4 Real-time exposure: the per-tick calculation

Current exposure to a counterparty is the netted, non-negative mark-to-market of the trades facing it: `CE_c = max(Σ_{t∈c} MtM_t, 0)`. For a linear instrument `MtM_t = notional_t · (price − trade_price_t)`. The naïve approach reprices the whole book on every tick; the scalable approach reprices **only the trades on the instrument that ticked**, and folds the delta into a running per-counterparty exposure — O(trades-on-that-instrument), not O(book).

ALGORITHM — T0: on each committed price tick `(instrument I, price p)`

```
1  Δ ← p − last_price[I]
2  pnl_window.push( Δ · Σ_{t on I} notional_t )        ▷ portfolio P&L for VaR
3  for each trade t on I:
4      mtm′ ← notional_t · (p − trade_price_t)
5      exposure[t.cp] += mtm′ − mtm[t];  mtm[t] ← mtm′   ▷ incremental netting
6  for each affected counterparty c:
7      if max(exposure[c], 0) > limit[c]:  alert(c, LIMIT_BREACH)
8  last_price[I] ← p
```

This runs on every committed tick, at ingest speed — the change stream delivers the tick the instant it commits.

15.5 Portfolio & systemic risk: the periodic pass

The heavy analytics run on a cadence over a consistent `Graph::view()` snapshot, off the write path.

Value-at-Risk. A 99% one-tick VaR by historical simulation: the worst 1% loss in the rolling P&L window. A breach of the VaR limit raises a portfolio alert.

Systemic importance. `pagerank` over the liability network. Because edges run debtor → creditor, PageRank mass accumulates at the entities everyone owes — the hubs whose failure would ripple furthest.

Default contagion — the Eisenberg–Noe clearing vector. The centerpiece. Given external assets `e_i` (each entity's outside cash) and the liability edges, the clearing payment vector `p` solves

$$p_i = \min\Big(\bar{p}_i,\; e_i + \sum_j \Pi_{ji}\, p_j\Big), \qquad \bar{p}_i = \sum_j L_{ij}, \quad \Pi_{ji} = L_{ji}/\bar{p}_j.$$

Each node pays the smaller of what it owes and what it actually has once upstream payments arrive. ChakraDB solves it by the monotone Picard iteration from `p = p̄` downward — which converges to

the greatest clearing vector — and reports each entity's payment, surviving equity, and, crucially, the set of **defaulters**: exactly who a cascade sinks. Feed it a stressed `external_assets` (one entity's cash set to zero) and it simulates the shock's contagion.

ALGORITHM — T2: periodic systemic pass over one snapshot

```
1  view ← exposures.view()
2  if VaR₉₉(pnl_window) > var_limit:  alert(PORTFOLIO, VAR_BREACH)
3  hub ← argmax view.pagerank();       alert(hub, SYSTEMICALLY_IMPORTANT)
4  for d in view.eisenberg_noe(external_assets).defaulted:
5      alert(d, DEFAULT_CASCADE)
6  for ring in view.laundering_cycles():
7      for m in ring: alert(m, CIRCULAR_EXPOSURE)
```

15.6 The worker is a materialized worker

The whole risk engine is one materialized worker — a named, incrementally-maintained derivation of the data. In Rust it implements `MaterializedWorker` and is registered on the change stream:

```rust
let ccr = cdc.materialize(Some("market_data"), RiskWorker::new(engine, exposures,
↪  trades, assets));
// ... the feed streams; the worker maintains exposure and fires alerts ...
ccr.query(|w| w.cp_exposure.clone());   // real-time exposure, any time
ccr.stop();                             // client owns the lifecycle
```

In Python it is a class plus `conn.on_change` — the same design, the full engine (SQL, exact money, the graph algorithms, `eisenberg_noe`) in a few hundred lines:

```python
def react(old, new):
    worker.on_change(old, new)          # dict: {"instrument":…, "price":…}
conn.on_change("market_data", react)
```

15.7 Capacity

The shipped pipeline (`examples/ccr_pipeline.rs`) runs the feed and the reacting risk worker concurrently over one engine:

```
Ingest: 40,000 market-data ticks in 0.84s
        = 47,790 ticks/s  ≈  172 million/hour
All injected risks detected live — CCR pipeline verified.
```

Real-time exposure, single-name limits, VaR, systemic-importance, default-cascade contagion, and circular-exposure detection — all live, over one consistent dataset, while the feed keeps ticking. The periodic Eisenberg–Noe and PageRank passes run on an MVCC snapshot, so they never stall ingest. That non-blocking split is the difference between real-time risk and an overnight batch.

15.8 What ChakraDB made possible

Risk measure	ChakraDB primitive
Real-time current exposure & netting	incremental per-tick repricing (T0)
Single-name limit monitoring	live exposure vs `limits`
Portfolio VaR / Expected Shortfall	historical simulation over the P&L window
Systemic importance	`pagerank` on the exposure network
Default-cascade contagion	**`eisenberg_noe`** clearing vector
Circular exposures / netting	`laundering_cycles` (SCC)
Netting sets	`connected_components`

The injected risks — a concentrated single-name breach, a portfolio VaR breach, a systemic hub, a thinly-capitalised default cascade, and a ring of circular exposures — are every one detected **live**, in one embedded process, while the market-data feed streams at 170 million ticks/hour. **Risk, in real time, over one consistent view of the book.**

Chapter 16

Case Study: Real-Time Recommendations

People who liked this also liked …

— every store, everywhere

The first two case studies catch *bad* behaviour. This one is the opposite: it helps, and it shows ChakraDB's reach beyond fraud and risk. A stream of user interactions — views, clicks, purchases — drives a **live recommendation engine**: as engagement arrives, "what should this user see next?" is answered in real time, over one consistent graph, never blocking the ingest that feeds it.

The workload usually needs a separate feature store, a nightly model-training job, and a serving tier reconciled by pipelines. Here it is one embedded process reacting to the interaction stream. The runnable code is `examples/reco_pipeline.rs` (Rust) and `examples/reco_stream.py` (Python).

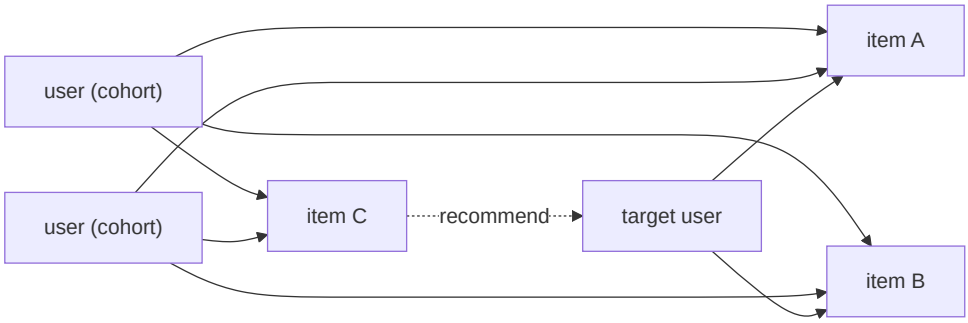
16.1 Recommendation is link prediction on a bipartite graph

Model engagement as a bipartite graph: users on one side, items on the other, an edge for each interaction. "What should user u see?" becomes *link prediction* — which edges are likely to form next — and ChakraDB has the primitives built in:

Question	Built-in primitive
What should this user see next?	`recommend(user, k)` — random-walk-with-restart
You might also like …	`adamic_adar(a, b)` — rarity-weighted item similarity
What's trending?	`out_degree` / `pagerank`
Users like this one	`jaccard_similarity`, `common_neighbors`

Edges are added **both ways** (`u → i` and `i → u`) so a random walk can bounce user → item → user → item, which is exactly collaborative filtering: the walk reaches items engaged by users similar to

you.



The target user engaged A and B; a cohort engaged A, B **and C**; the walk from the target reaches C through the cohort — so C is recommended.

16.2 The schema

```
CREATE TABLE interactions (id INTEGER PRIMARY KEY, user_id INTEGER,
    item_id INTEGER, kind VARCHAR(8), ts TIMESTAMP);   -- the engagement stream
```

```
CREATE TABLE recommendations (id INTEGER PRIMARY KEY, user_id INTEGER, item_id INTEGER);
```

The engagement **graph** is derived from the stream; item nodes live above the user-id range so the two never collide.

16.3 The recommender is a materialized worker

The engine is one materialized worker over the interaction stream. Each interaction folds into the graph (T0); recommendations refresh periodically over a snapshot (T2).

ALGORITHM — T0: on each committed interaction `(user u, item i)`

```
1  graph.add_edge(u, item_node(i));  graph.add_edge(item_node(i), u)   ▷ both ways
```

ALGORITHM — T2: refresh recommendations over one snapshot

```
1  view ← graph.view()
2  recs    ← view.recommend(u, k)            ▷ items reached via similar users,
 ↪  unseen
3  trending← items sorted by view.out_degree ▷ most distinct users
4  similar ← view.adamic_adar(anchor, ·)     ▷ "you might also like"
```

`recommend` runs personalized PageRank seeded at the user, drops the user's already-engaged items and any zero-signal candidate, and returns the top-`k` remainder — collaborative filtering expressed as a graph walk, in one call.

16.4 In code

Rust — register the recommender on the interaction stream:

```rust
let engagement = Graph::open(db.clone(), "engagement")?;
let reco = cdc.register("recommender", Some("interactions"),
                        Recommender::new(engine.clone(), engagement));
// … the feed streams; recommendations refresh incrementally …
reco.query(|w| w.last_recommendations.clone());
```

Python — the same worker as a class plus `conn.on_change`:

```python
view = graph.view()
recs   = view.recommend(target_user, 10)      # [(item_node, score), …]
trend  = sorted(items, key=view.out_degree, reverse=True)
also   = view.adamic_adar(item_a, item_b)
```

16.5 Capacity

```
Ingest: 50,000 interactions in 1.1s
        = 45,899 interactions/s  ≈  165 million/hour
Recommendations verified — the cohort's item surfaced live.
```

Recommendations stay fresh as engagement streams, because the heavy walk runs on an MVCC snapshot off the write path. A cohort's co-engaged item is surfaced to the target user; the trending

item ranks first by engagement; Adamic–Adar ranks the bundle items as most similar — every check passing on the live stream.

16.6 The pattern, three times over

This is the third system built on the same machinery — the change stream, a materialized worker, and the graph library — as AML and Counterparty Risk. Catch laundering, measure systemic risk, or personalize a feed: the engine is the same, the workload reacts to live data in one process, and the reader never blocks the writer. That is what makes ChakraDB more than a query engine — it is a substrate for real-time systems.

Chapter 17

Case Study: Real-Time Fraud Detection (HTAP + Graph)

Trust, but verify.

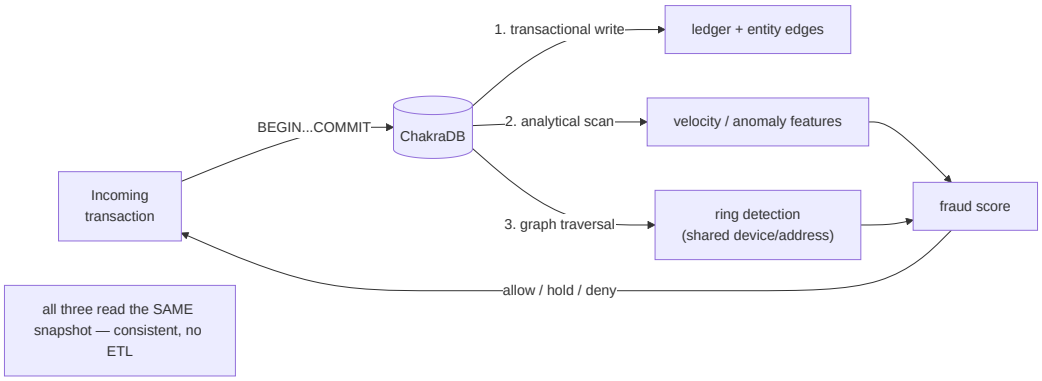
— *Russian proverb*

This case study exercises everything at once: transactional ingest, analytical scoring, and graph traversal — over **one consistent snapshot**, in one embedded process. It is the workload ChakraDB is built for.

17.1 The problem

A payments service must, for each incoming transaction, decide *in-line* whether it looks fraudulent. Fraud signals are both **statistical** (this card's spend rate just spiked) and **relational** (this device has touched five accounts that share a shipping address with a known mule). Traditionally that means two or three systems: an OLTP store for the writes, a warehouse for the stats, and a graph database for the relationships — kept in sync by ETL, always a little stale.

ChakraDB collapses them.



17.2 The schema

```
-- The transactional ledger (exact money — no rounding).
CREATE TABLE txns (
  id        INT PRIMARY KEY,
  card_id   INT NOT NULL,
  device_id INT NOT NULL,
  amount    DECIMAL(12,2) NOT NULL CHECK (amount >= 0),
```

```
  ts        TIMESTAMP NOT NULL,
  status    VARCHAR(8) DEFAULT 'pending'
);

-- The entity graph: cards, devices, addresses linked by shared use.
-- (Managed by the Graph handle; edges keyed (src,dst) src-major.)
```

Nodes are entities (cards, devices, addresses, accounts); an edge means "these two entities were seen together." A fraud *ring* is a dense cluster in this graph.

17.3 Ingest: the transactional write

Each transaction is one ACID transaction — the ledger row and the entity edges commit together, so no reader ever sees half of it:

```
BEGIN;
  INSERT INTO txns VALUES (9001, 55, 88, 249.99, '2026-07-22 14:03:01', 'pending');
  -- link card 55 <-> device 88 in the entity graph (via the Graph handle)
COMMIT;                          -- first-committer-wins; crash-atomic
```

`DECIMAL(12,2)` keeps money exact; `CHECK`/`NOT NULL` reject bad rows before they touch the log.

17.4 Scoring: analytical + graph, one snapshot

The scorer takes a single snapshot and reads all three signals from it — no cross-system consistency to reason about:

```
let snap_engine = &engine;                  // SQL over the current snapshot
let graph_view  = fraud_graph.view()?;      // CSR over the SAME instant

// (A) Statistical: this card's velocity in the last minute.
let velocity: i64 = snap_engine
    .query("SELECT COUNT(*) FROM txns \
            WHERE card_id = 55 AND ts > '2026-07-22 14:02:01'")?
    [0][0].parse()?;

// (G) Relational: how tightly is this device tied into a cluster?
let ring_size = graph_view
    .connected_components_of(device_node)   // its component
    .len();
let local_density = graph_view.triangle_count_around(device_node);

let score = weight_v * velocity as f64
          + weight_r * (ring_size as f64) * (1.0 + local_density as f64);
```

The analytical `COUNT(*)` runs on the interpreter with zonemap pruning; the graph traversal runs over the CSR view. **Both see the transaction that was just committed** — because they read the same MVCC snapshot as the ledger.

17.5 Why the other architectures struggle here

- **Warehouse + ETL:** the graph and stats lag the ledger by minutes — useless for in-line scoring.
- **Separate graph DB:** the scorer must join across two systems and reconcile two consistency models; the graph mutation contends with the traversal.
- **Single-writer OLAP (DuckDB):** cannot ingest concurrently with the scan; the write stream would have to pause for every analytical read.

ChakraDB serves all three from one snapshot, in one process, with writers never blocked by the scorer.

17.6 Operating it

- Run the **scorer** in-line (per transaction) using live adjacency (`out_neighbors`, `out_degree`) for cheap signals, and a periodic `view()` + `pagerank`/`connected_components` pass for the expensive ring analysis (see Live Graph Analytics).
- Watch `Storage::stats()` for checkpoint lag and backpressure under peak ingest (see Observability).
- Back up the store with `backup_to` on a cadence (see Backup & Restore).

17.7 The takeaway

Fraud detection is the archetypal **HTAP + graph** workload: fast writes, live statistics, and relationship traversal, all needing to agree on the *same instant*. ChakraDB's contribution is that the agreement is free — it is one snapshot clock — so the application is a few queries and a graph view, not a three-system integration project.

Part VIII

Perspective

Chapter 18

ChakraDB vs. DuckDB

If you know the enemy and know yourself, you need not fear the result of a hundred battles.

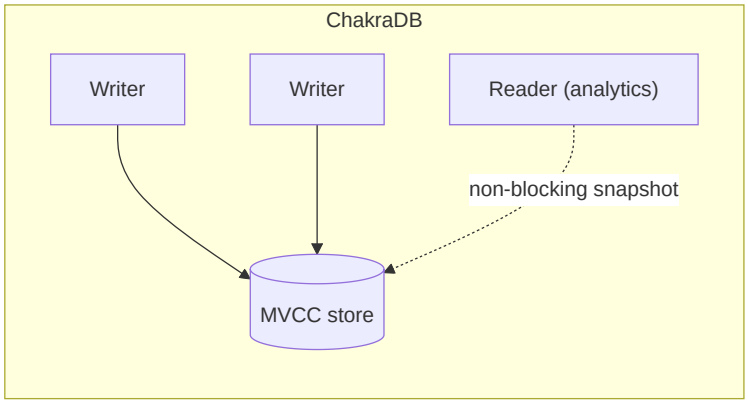
— *Sun Tzu*

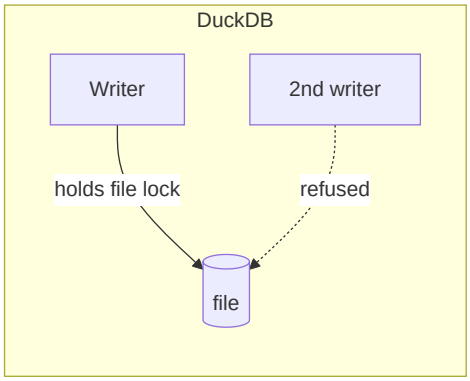
DuckDB is the reference point — the best-in-class embedded analytical engine. This chapter is honest about where DuckDB wins, where ChakraDB wins, and the one axis where the comparison is *structural* rather than a matter of tuning.

18.1 The structural difference: concurrent writers

DuckDB holds a **single-writer file lock**. A second process opening the same database for writing is refused at the OS level (`IO Error: Could not set lock`). That is not a tuning parameter — it is the concurrency model.

ChakraDB permits **continuous concurrent writers with non-blocking snapshot reads**. This is the wedge: ChakraDB serves a workload DuckDB *structurally cannot* — a live write stream with analytics running over it at the same time.





Measured: under 4 threads issuing durable, WAL-logged writes, ChakraDB's analytical query p50 degrades just **1.49×** (2.2 → 3.3 ms) while ~8,900 writes commit *during* the measurement, readers never block, and every query sees a stable snapshot (`wedge-bench`). DuckDB cannot run this benchmark — the second writer is refused.

18.2 Cold-scan analytics: DuckDB's home turf

On a static dataset scanned cold, DuckDB is excellent and mature. ChakraDB does not try to outscan it; the goal is to be *competitive* while adding concurrency. On a 105-column ClickBench-shaped table, identical results to DuckDB, p50 ms:

Query	10M — ChakraDB	10M — DuckDB	Winner
`COUNT(*)`	0.0	0.0	tie
filtered count (`<>`)	4.8	1.0	DuckDB
sum/avg	5.3	3.0	DuckDB
`COUNT(DISTINCT UserID)`	**40.6**	60.0	**ChakraDB 1.5×**
`COUNT(DISTINCT SearchPhrase)`	**12.0**	36.0	**ChakraDB 3×**
min/max	0.1	0.0	tie (both instant)
group-by adv engine	4.9	2.0	DuckDB
top users (group+sort)	**59.7**	71.0	**ChakraDB**
top phrases / by time	55.7 / 52.1	24.0 / 15.0	DuckDB
pk range ~20 rows	1.1	1.0	tie (both prune)

Reading it: - **ChakraDB wins the big `COUNT(DISTINCT)`s at scale** (up to 3×) and top-users — and those margins *widen* as rows grow. - **DuckDB wins simple `GROUP BY` and top-K** — its vectorized hash-agg and top-K operators are more optimized, and zonemaps don't help those shapes. - **Selective range scans are a tie** — both prune (DuckDB via rowgroup stats, ChakraDB via zonemaps); a needle range stays ~1 ms as the table grows 100×.

The full harness (both engines, identical CSV) is `src/bin/clickbench.rs` + `scripts/clickbench_duckdb.sh`; the numbers and method are in Benchmark Methodology.

18.3 Feature and model comparison

	DuckDB	ChakraDB
Model	embedded OLAP	embedded **HTAP + graph**
Concurrent writers	× single-writer lock	MVCC, non-blocking reads
Transactions		(BEGIN/COMMIT/ROLLBACK, first-committer-wins)
Analytical engine	native vectorized	DataFusion (vectorized)
Graph	×	built-in (adjacency + algorithms)
On-disk format	native (open via extensions)	Arrow IPC parts (open)
Exact `DECIMAL`		(i128 / Arrow Decimal128)
Backup/restore	file copy	`backup_to` / `restore` (consistent)
Maturity	very high	young, but crash-tested

18.4 When to pick which

- **Pick DuckDB** for the fastest cold-scan analytics over data you loaded earlier, on a laptop-class machine, with a single writer.
- **Pick ChakraDB** when writes and analytics happen *at the same time* on live data, when you also need **graph** traversals over that data, or when concurrent ingest must never pause for a reader.

They are not the same product. DuckDB optimizes the *static-analytics* axis; ChakraDB optimizes the *live, concurrent, multi-model* axis.